

Task Questions

1. Question 1

A monolithic architecture is a classic software development paradigm where several business tasks are carried out by a single code base. Because of the data interchange protocols built into the system, every software component in a monolithic system depends on every other component. Monolithic architecture is difficult and time-consuming to modify since even minor adjustments affect a significant portion of the code base.

Microservices, on the other hand, are an architectural style that divides software into discrete, stand-alone parts or services. Every service has a particular purpose and interacts with other services via an established interface. You may update, change, deploy, or scale each service as needed because they each operate separately.

2. Question 2

With the help of technologies like virtual machines and containers, our apps can operate independently of the resources in your IT infrastructure. A software code bundle that includes the code, libraries, and other requirements for an application is called a container. Our apps become portable thanks to containerization, enabling the same code to run across many platforms.

A digital copy of a physical machine is known as a virtual machine. On the same host operating system, you can run many virtual machines, each with its own operating system. You can also build a virtual machine with all the components needed to run your program on it.

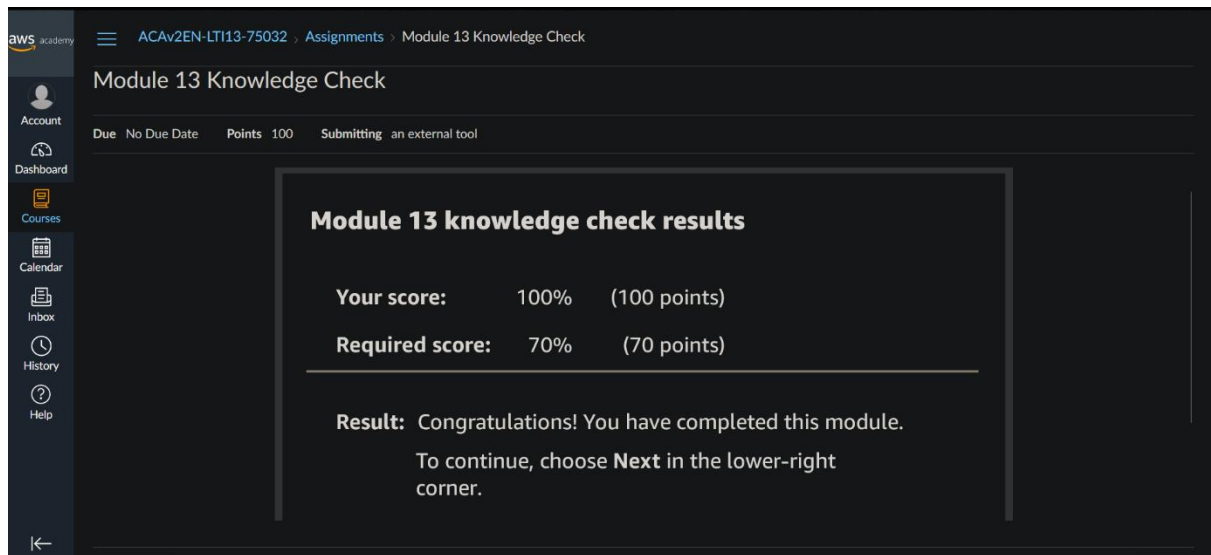
3. Question 3

`docker build -t mb-repo .`

Module 13 Knowledge check

This knowledge check was comprised of the most important questions throughout the module 13. The questions asked was kind of a summary to the whole module 13. In this knowledge check I was able to see my understanding of this module, and the from the mistakes I was able to correct myself of the parts that I was not sure about.

It was mainly about microservice architecture. the first question itself was about their characteristics. then there were questions about containers, Amazon ECS. then there were questions about AWS Fargate, that it is a service that enables us to run containers without needing to manage servers to clusters. then there were questions about serverless architectures and their benefits. Then there were questions about scaling in lambda functions, and finally some real life situations and scenarios.



The screenshot shows the AWS Academy interface for the 'Module 13 Knowledge Check'. The left sidebar contains navigation links: Account, Dashboard, Courses, Calendar, Inbox, History, and Help. The main content area displays the following information:

- Module 13 Knowledge Check**
- Due:** No Due Date
- Points:** 100
- Submitting:** an external tool

Module 13 knowledge check results

Your score:	100%	(100 points)
Required score:	70%	(70 points)

Result: Congratulations! You have completed this module.
To continue, choose **Next** in the lower-right corner.

Module 12 Knowledge check

This knowledge check was comprised of the most important questions throughout the module 12. The questions asked was kind of a summary to the whole module 12. In this knowledge check I was able to see my understanding of this module, and the from the mistakes I was able to correct myself of the parts that I was not sure about.

This knowledge check was mainly about Decoupled Architectures. The first question itself was asking about the difference between tightly and loosely coupled architectures. There were questions about Amazon Queue Service (Amazon SQS), like where and how we should use it. Then there were questions about Amazon Notification Service (Amazon SNS), like what are its use cases, and features. Then there were questions about Amazon MQ, about its use cases and finally a real-life scenario about using the Lambda Functions.

The screenshot shows the AWS Academy interface for the 'Module 12 Knowledge Check'. On the left is a sidebar with navigation links: Account, Dashboard, Courses, Calendar, Inbox, History, and Help. The main content area has a dark background. At the top, it says 'Module 12 Knowledge Check' with a status bar showing 'Due No Due Date', 'Points 100', and 'Submitting an external tool'. Below this, a box titled 'Module 12 knowledge check results' displays the following information:

Your score:	90%	(90 points)
Required score:	70%	(70 points)

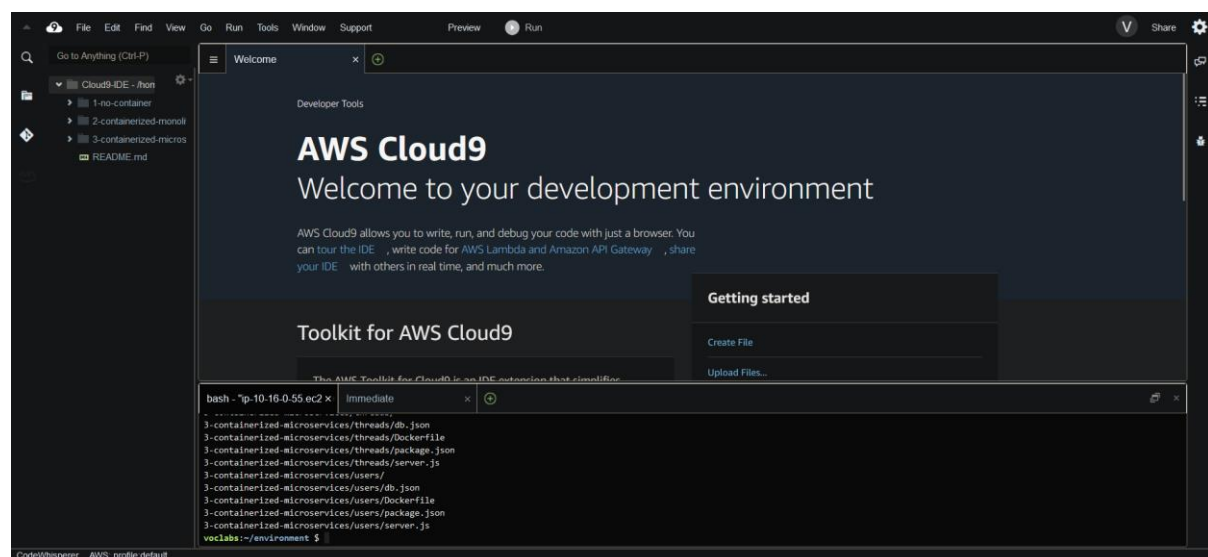
Result: Congratulations! You have completed this module.
To continue, choose **Next** in the lower-right corner.

At the bottom of the interface, there are two buttons: 'Previous' on the left and 'Next' on the right.

Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Task 1: Preparing the development environment

I start by opening the cloud-based integrated programming environment (IDE) called AWS Cloud9 IDE. I open the environment by copying the Cloud9 URL from the AWS Details section and pasting it into a new browser tab. Next, I use the given command to download and extract the necessary lab files. After extraction, the IDE's left pane displays the directories 1-no-container, 2-containerized-monolith, and 3-containerized-microservices. Different versions of the application for different deployment contexts are contained in these folders. Since I'll be using the AWS Cloud9 IDE tab a lot during the lab, I keep it open.

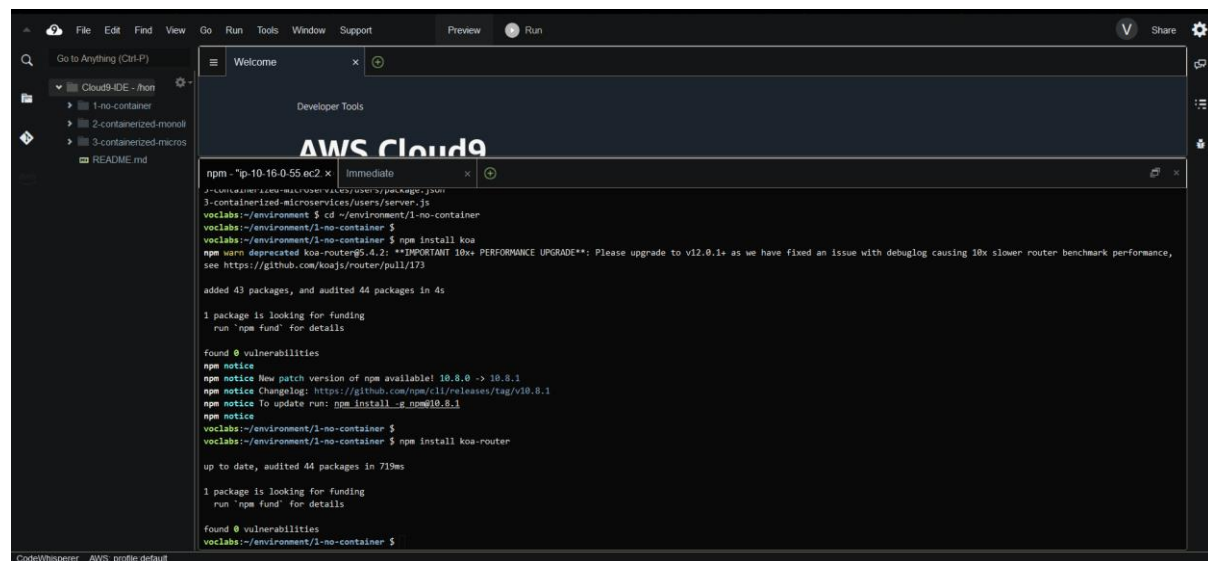


Application into Microservices

Task 2: Running the application on a basic Node.js server

Task 2.1: Installing the required Node.js modules

I must first install the Node.js modules required for the application, which makes use of the Koa framework. I launch the AWS Cloud9 IDE terminal and use the command `cd ~/environment/1-no-container` to go to the directory that holds the application files. Next, I use the commands `npm install koa` and `npm install koa-router` to install the koa and koa-router modules. The output from the console shows that the installation is proceeding as planned and that the modules have been downloaded and are now in the 1-no-container/node_modules folder in the AWS Cloud9 environment.



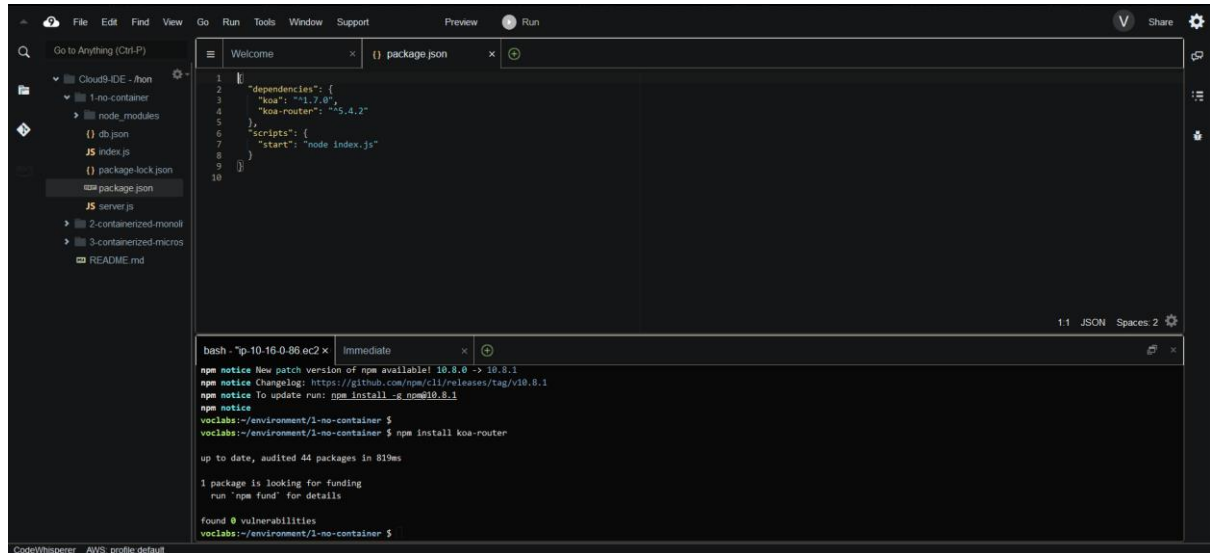
The screenshot shows the AWS Cloud9 IDE interface. The left sidebar displays the file explorer with a tree view containing 'Cloud9-IDE - /home', '1-no-container', '2-containerized-monolith', '3-containerized-microservices', and 'README.md'. The main editor area shows a terminal window with the following output:

```
npm - "p-10-16-0-55 ec2 x" Immediate x
$ cd ~/environment/1-no-container
voclabs:~/environment $ cd ~/environment/1-no-container
voclabs:~/environment/1-no-container $ npm install koa
npm warn deprecated koa-router@5.4.2: **IMPORTANT 10x PERFORMANCE UPGRADE**: Please upgrade to v12.0.1+ as we have fixed an issue with debuglog causing 10x slower router benchmark performance, see https://github.com/koajs/router/pull/173
added 43 packages, and audited 44 packages in 4s
1 package is looking for funding
  run `npm fund` for details
found 0 vulnerabilities
npm notice
npm notice New patch version of npm available! 10.8.0 -> 10.8.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v10.8.1
npm notice To update run: npm install -g npm@10.8.1
npm notice
voclabs:~/environment/1-no-container $ npm install koa-router
up to date, audited 44 packages in 719ms
1 package is looking for funding
  run `npm fund` for details
found 0 vulnerabilities
voclabs:~/environment/1-no-container $
```

Application into Microservices

Task 2.2: Reviewing the application design and code

I begin by looking at the package.json file, noticing that index.js is the entry point and that koa and koa-router dependencies are listed.



The screenshot shows the Visual Studio Code editor with the `package.json` file open. The file contains the following JSON:

```
{
  "dependencies": {
    "koa": "^1.7.0",
    "koa-router": "^5.4.2"
  },
  "scripts": {
    "start": "node index.js"
  }
}
```

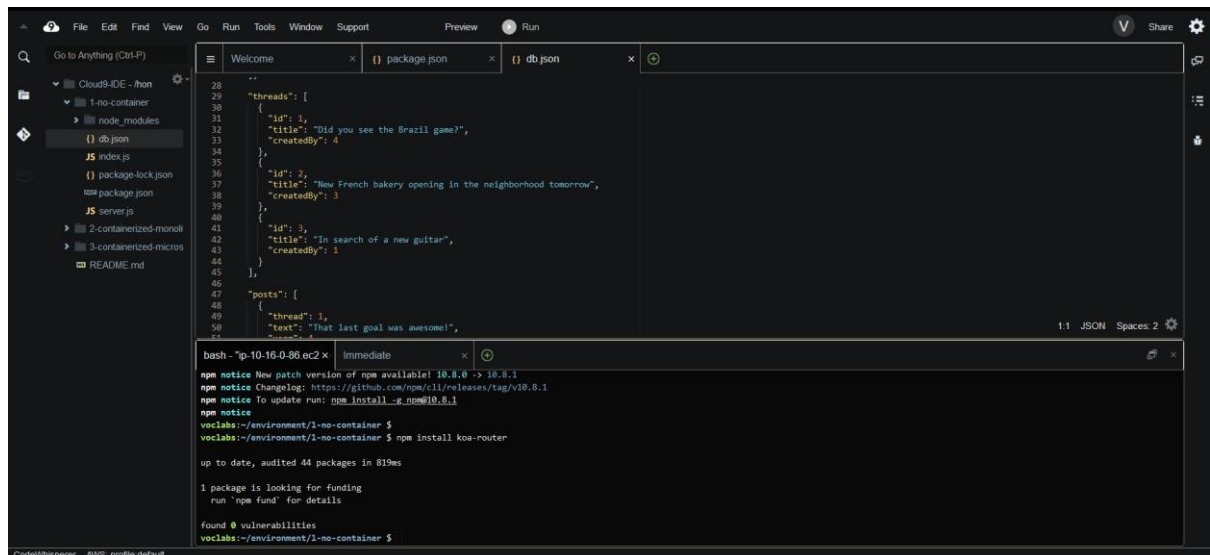
The terminal window at the bottom shows the output of running `npm install` in a container:

```
bash - "ip-10-16-0-86 ec2" x Immediate x
npm notice New patch version of npm available! 10.8.0 -> 10.8.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v10.8.1
npm notice To update run: npm install -g npm@10.8.1
npm notice
voclabs:~/environment/1-no-container $
voclabs:~/environment/1-no-container $ npm install koa-router
up to date, audited 44 packages in 819ms

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
voclabs:~/environment/1-no-container $
```

I then go over `db.json`, which has an overview of sample posts, threads, and users. Thread titles include "Did you see the Brazil game?" and "New French bakery opening in the neighborhood tomorrow." Users have names like Marcerline Singer and Finn Alberts. Messages connected to these threads and users are listed in the posts property.

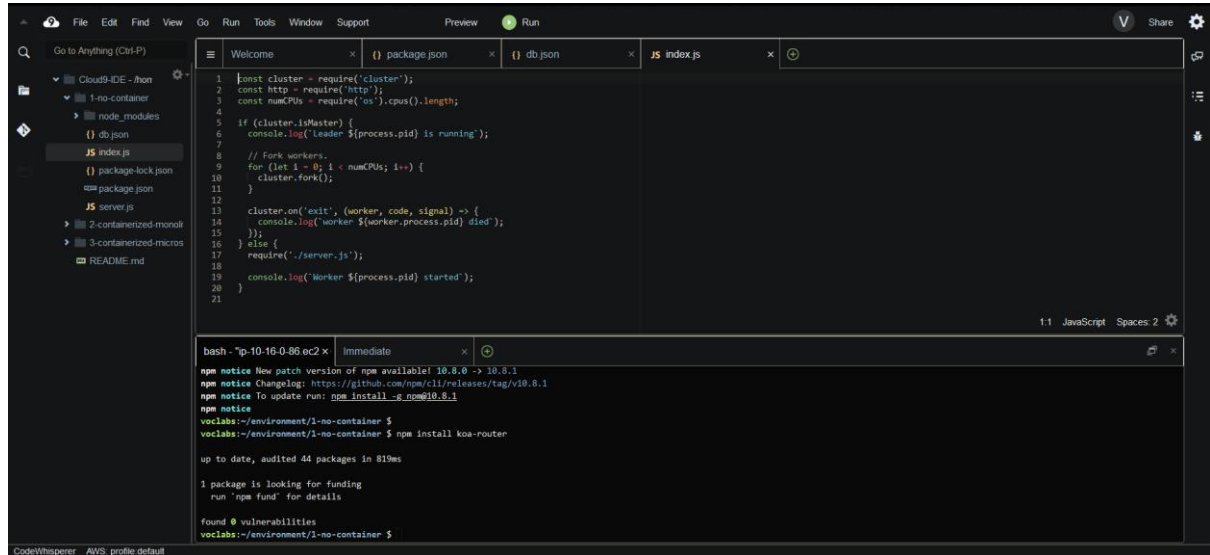


The screenshot shows the Visual Studio Code editor with the `db.json` file open. The file contains the following JSON:

```
{
  "threads": [
    {
      "id": 1,
      "title": "Did you see the Brazil game?",
      "createdBy": 4
    },
    {
      "id": 2,
      "title": "New French bakery opening in the neighborhood tomorrow",
      "createdBy": 3
    },
    {
      "id": 3,
      "title": "In search of a new guitar",
      "createdBy": 1
    }
  ],
  "posts": [
    {
      "thread": 1,
      "text": "That last goal was awesome!",
      "createdBy": 4
    }
  ]
}
```

The terminal window at the bottom shows the same output as the previous screenshot, indicating that the `npm install` command was run successfully.

I then examine index.js, noting that it imports the http, os, and cluster modules, determines the CPU core count, and uses the Node.js cluster feature to create worker threads for every core.

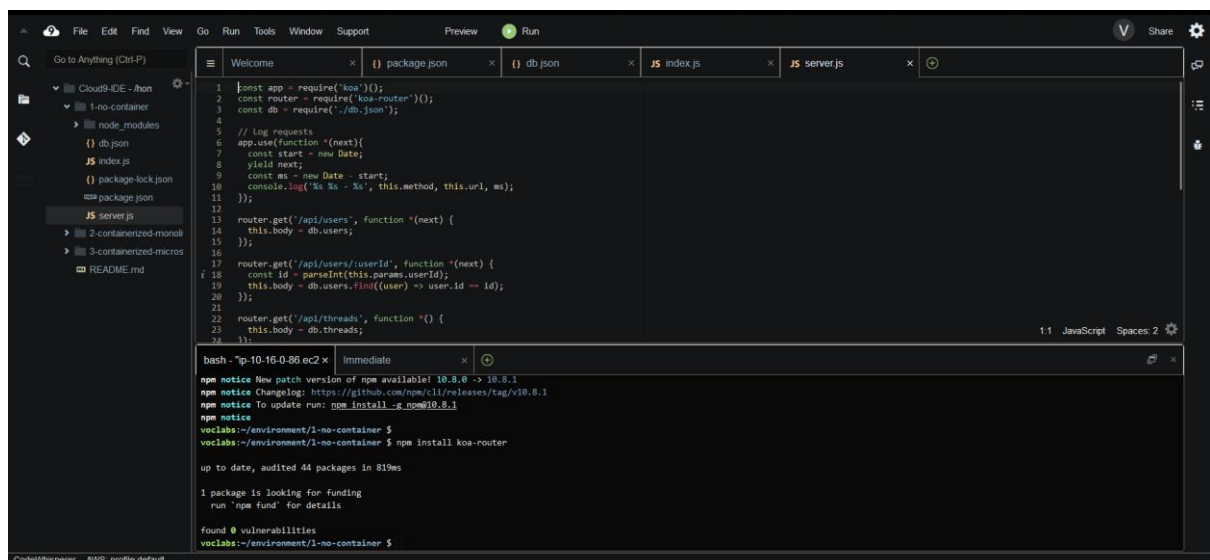


The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the content of `index.js`, which uses the `cluster` module to fork workers based on the number of CPU cores. The terminal at the bottom shows the output of running `npm install`, including notices about new versions of npm and the results of the installation process.

```
1 const cluster = require('cluster');
2 const http = require('http');
3 const numCPUs = require('os').cpus().length;
4
5 if (cluster.isMaster) {
6   console.log(`Leader ${process.pid} is running`);
7
8   // Fork workers.
9   for (let i = 0; i < numCPUs; i++) {
10    cluster.fork();
11  }
12
13  cluster.on('exit', (worker, code, signal) => {
14    console.log(`Worker ${worker.process.pid} died`);
15  });
16 } else {
17   require('./server.js');
18   console.log(`Worker ${process.pid} started`);
19 }
20
21
```

```
bash - "p-10-16-0-86 ec2" x Immediate x
npm notice New patch version of npm available! 10.8.0 -> 10.8.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v10.8.1
npm notice To update run: npm install -g npm@10.8.1
npm notice
voclabs:~/environment/1-no-container $
voclabs:~/environment/1-no-container $ npm install koa-router
up to date, audited 44 packages in 819ms
1 package is looking for funding
run 'npm fund' for details
found 0 vulnerabilities
voclabs:~/environment/1-no-container $
```

Lastly, I go over `server.js`, starting with the establishment of a generator function to log request details and the import of `db.json`. Several RESTful API methods are defined in the file, including getting user or thread collections and obtaining particular postings by user or thread ID.



The screenshot shows the VS Code editor with the file explorer on the left. The main editor window displays the content of `server.js`, which sets up a Koa application with a router for `/api/users` and `/api/threads`. The terminal at the bottom shows the same `npm install` output as the previous screenshot.

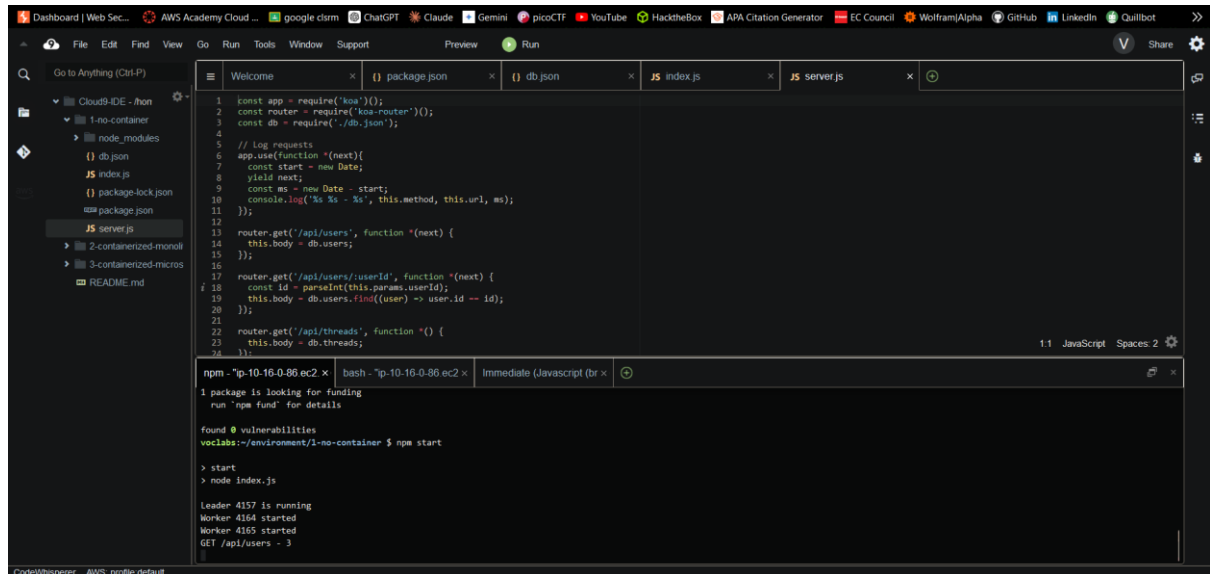
```
1 const app = require('koa')();
2 const router = require('koa-router')();
3 const db = require('./db.json');
4
5 // Log requests
6 app.use(function (next) {
7   const start = new Date;
8   yield next;
9   const ms = new Date - start;
10  console.log(`%s %s - %s`, this.method, this.url, ms);
11 });
12
13 router.get('/api/users', function (next) {
14   this.body = db.users;
15 });
16
17 router.get('/api/users/:userId', function (next) {
18   const id = parseInt(this.params.userId);
19   this.body = db.users.find((user) => user.id == id);
20 });
21
22 router.get('/api/threads', function () {
23   this.body = db.threads;
24 });
25
```

```
bash - "p-10-16-0-86 ec2" x Immediate x
npm notice New patch version of npm available! 10.8.0 -> 10.8.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v10.8.1
npm notice To update run: npm install -g npm@10.8.1
npm notice
voclabs:~/environment/1-no-container $
voclabs:~/environment/1-no-container $ npm install koa-router
up to date, audited 44 packages in 819ms
1 package is looking for funding
run 'npm fund' for details
found 0 vulnerabilities
voclabs:~/environment/1-no-container $
```

Application into Microservices

Task 2.3: Running the application

I first use the terminal tab to run the command `npm start` to launch the application and the server. The application's entry point, `index.js`, initializes when the server boots up and creates two cluster threads, `Leader` and `Worker`, to manage requests effectively.



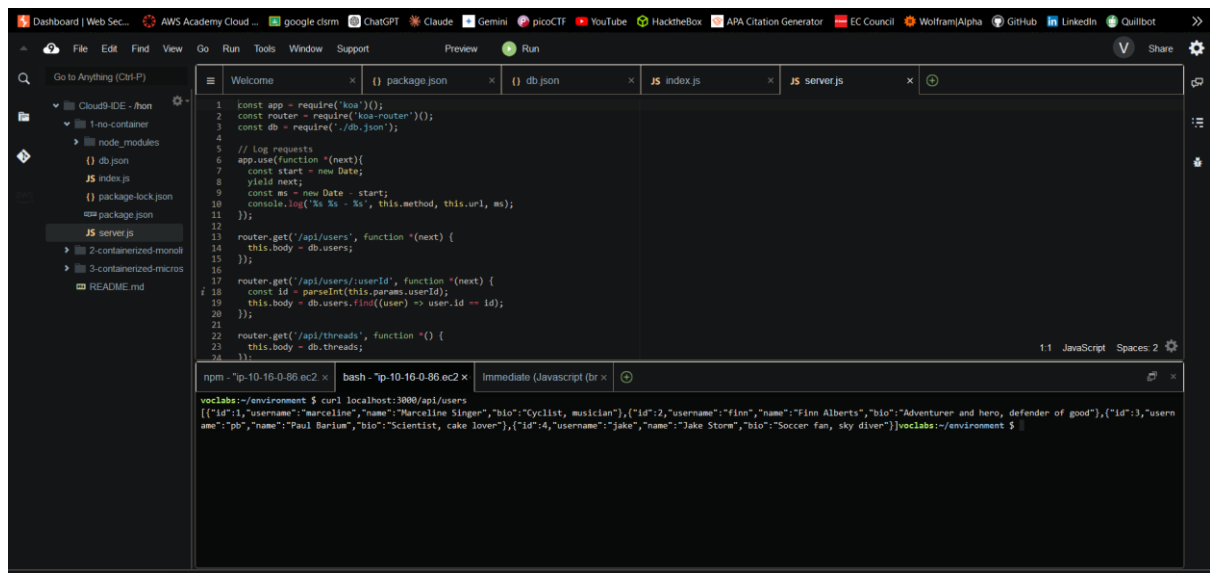
The screenshot shows the VS Code editor with the following files open: `package.json`, `db.json`, `index.js`, and `server.js`. The `index.js` file contains the following code:

```
1 const app = require('koa')();
2 const router = require('koa-router')();
3 const db = require('./db.json');
4
5 // Log requests
6 app.use(function (next) {
7   const start = new Date();
8   yield next;
9   const ms = new Date - start;
10  console.log(`%s %s - %s`, this.method, this.url, ms);
11 });
12
13 router.get('/api/users', function (next) {
14   this.body = db.users;
15 });
16
17 router.get('/api/users/:userId', function (next) {
18   const id = parseInt(this.params.userId);
19   this.body = db.users.find((user) => user.id == id);
20 });
21
22 router.get('/api/threads', function () {
23   this.body = db.threads;
24 });
```

The terminal output shows the following commands and results:

```
npm - "ip-10-16-0-86.ec2" x bash - "ip-10-16-0-86.ec2" x Immediate (Javascript (br x) x
1 package is looking for funding
  run "npm fund" for details
found 0 vulnerabilities
vocilabs:/environment/1-no-container $ npm start
> start
> node index.js
Leader 4157 is running
Worker 4164 started
Worker 4165 started
GET /api/users - 3
```

I then launch a new terminal tab and test the RESTful API of the program. I enter `curl localhost:3000/api/users` in the right terminal tab to access the `/api/users` resource. The message board database's user list is returned by this command as a JSON object. After that, I flip to the left terminal tab and view the `server.js` output message, which verifies that it successfully handled a GET method request on the `/api/users` path.



The screenshot shows the VS Code editor with the following files open: `package.json`, `db.json`, `index.js`, and `server.js`. The `index.js` file contains the following code:

```
1 const app = require('koa')();
2 const router = require('koa-router')();
3 const db = require('./db.json');
4
5 // Log requests
6 app.use(function (next) {
7   const start = new Date();
8   yield next;
9   const ms = new Date - start;
10  console.log(`%s %s - %s`, this.method, this.url, ms);
11 });
12
13 router.get('/api/users', function (next) {
14   this.body = db.users;
15 });
16
17 router.get('/api/users/:userId', function (next) {
18   const id = parseInt(this.params.userId);
19   this.body = db.users.find((user) => user.id == id);
20 });
21
22 router.get('/api/threads', function () {
23   this.body = db.threads;
24 });
```

The terminal output shows the following commands and results:

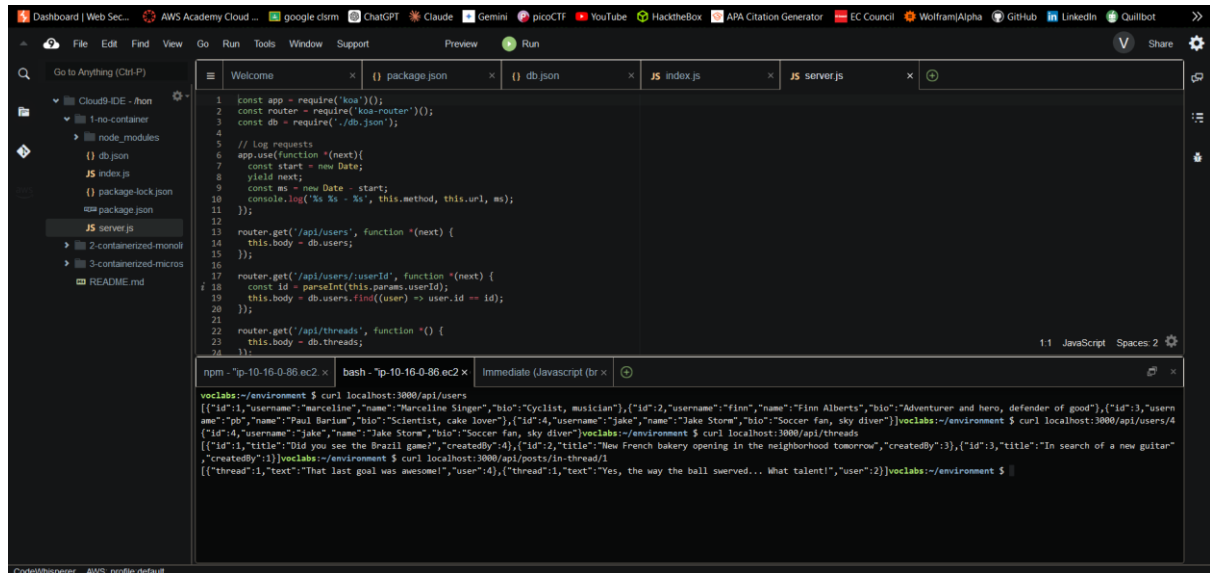
```
npm - "ip-10-16-0-86.ec2" x bash - "ip-10-16-0-86.ec2" x Immediate (Javascript (br x) x
vocilabs:/environment $ curl localhost:3000/api/users
[{"id":1,"username":"marcelline","name":"Marcelline Singer","bio":"Cyclist, musician"},{"id":2,"username":"flim","name":"Flim Alberts","bio":"Adventurer and hero, defender of good"},{"id":3,"username":"pb","name":"Paul Barium","bio":"Scientist, cake lover"},{"id":4,"username":"jake","name":"Jake Storm","bio":"Soccer fan, sky diver"}]
vocilabs:/environment $
```


SIT233 – Cloud Computing

Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

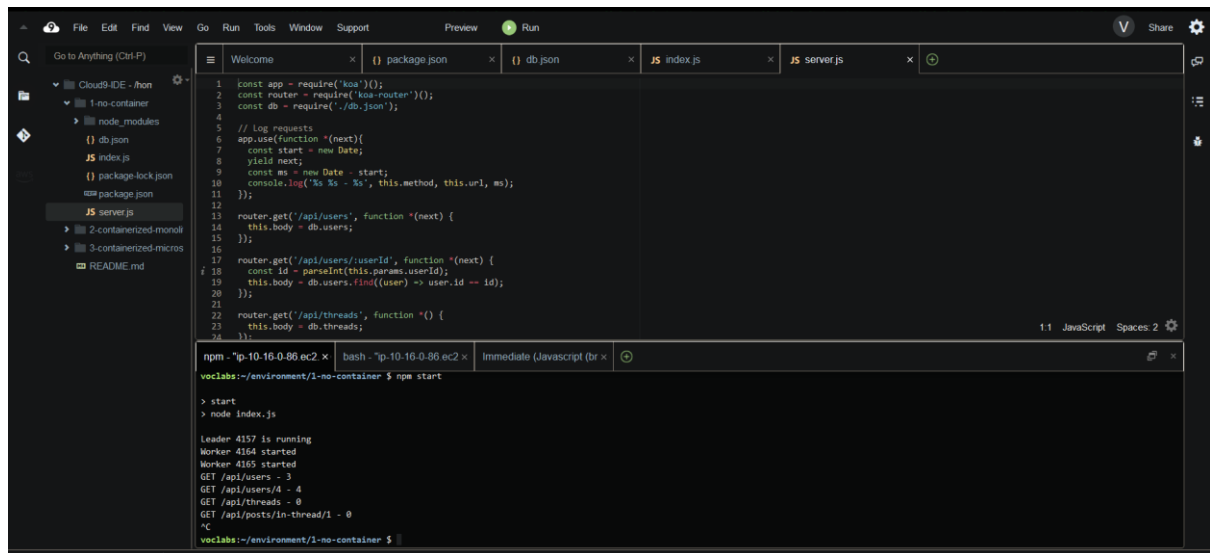
Analyzing the output and messages in the terminals, I also test various API calls to get data about users, threads, and postings. For instance, by typing `curl localhost:3000/api/users/4`, I may obtain facts about Jake Storm, the fourth user.



```
1 const app = require('koa')();
2 const router = require('koa-router')();
3 const db = require('./db.json');
4
5 // Log requests
6 app.use(function (next) {
7   const start = new Date();
8   yield next;
9   const ms = new Date - start;
10  console.log('%s %s - %s', this.method, this.url, ms);
11 });
12
13 router.get('/api/users', function (next) {
14   this.body = db.users;
15 });
16
17 router.get('/api/users/:userId', function (next) {
18   const id = parseInt(this.params.userId);
19   this.body = db.users.find((user) => user.id == id);
20 });
21
22 router.get('/api/threads', function () {
23   this.body = db.threads;
24 });
```

```
npm - "ip-10-16-0-86.ec2" x bash - "ip-10-16-0-86.ec2" x Immediate (Javascript (br x) x
voclabs:/environment $ curl localhost:3000/api/users
[{"id":1,"username":"Marceline Singer","bio":"Cyclist, musician"}, {"id":2,"username":"finn","name":"Finn Alberts","bio":"Adventurer and hero, defender of good"}, {"id":3,"username":"pb","name":"Paul Barium","bio":"Scientist, cake lover"}, {"id":4,"username":"jake","name":"Jake Storm","bio":"Soccer fan, sky diver"}]
voclabs:/environment $ curl localhost:3000/api/users/4
{"id":4,"username":"jake","name":"Jake Storm","bio":"Soccer fan, sky diver"}
voclabs:/environment $ curl localhost:3000/api/threads
[{"id":1,"title":"Did you see the Brazil game?","createdBy":4,"id":2,"title":"New French bakery opening in the neighborhood tomorrow","createdBy":3}, {"id":3,"title":"In search of a new guitar"}]
voclabs:/environment $ curl localhost:3000/api/posts/in-thread/1
[{"thread":1,"text":"That last goal was awesome!", "user":4}, {"thread":1,"text":"Yes, the way the ball swerved... What talent!", "user":2}]
voclabs:/environment $
```

After testing is finished, I ensure that the application responds to RESTful GET requests correctly by using the Ctrl+C keyboard shortcut in the left terminal tab.



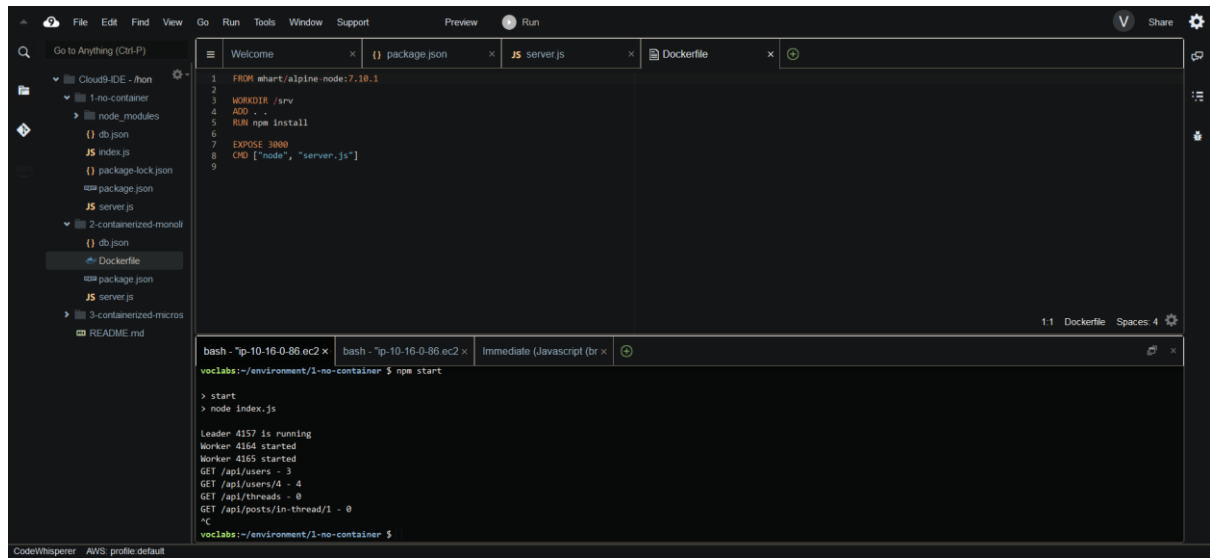
```
npm - "ip-10-16-0-86.ec2" x bash - "ip-10-16-0-86.ec2" x Immediate (Javascript (br x) x
voclabs:/environment/l1-no-container $ npm start
> start
> node index.js
Leader 4157 is running
Worker 4164 started
Worker 4165 started
GET /api/users - 3
GET /api/users/4 - 4
GET /api/threads - 0
GET /api/posts/in-thread/1 - 0
^C
voclabs:/environment/l1-no-container $
```

Application into Microservices

Task 3: Containerizing the monolith for Amazon ECS

Task 3.1: Preparing the application for Docker containerization

I go over the modifications done to the 2-containerized-monolith folder application. First, I open package.json and see that, in keeping with the elimination of clustering, the entry point is now server.js rather than index.js. After that, I see that server.js now has a line to output "Worker started" when the application is first starting up. I then take a look at the Dockerfile, which contains the construction instructions for the container image.



The screenshot shows a code editor with the following content:

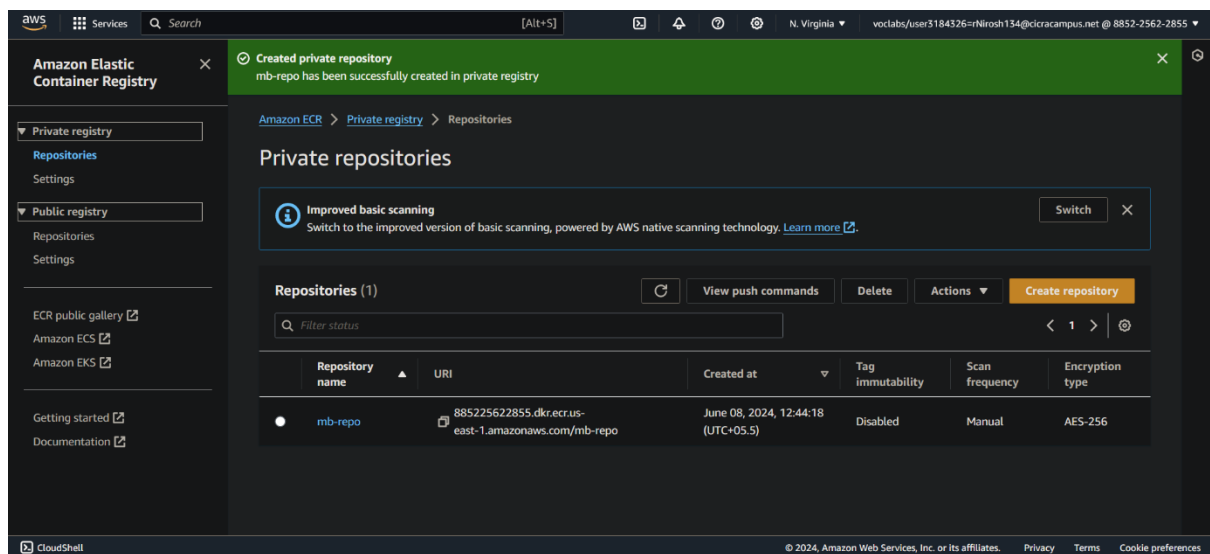
```
1 FROM alpine:node:7.10.1
2
3 WORKDIR /srv
4 ADD . .
5 RUN npm install
6
7 EXPOSE 3000
8 CMD ["node", "server.js"]
9
```

The terminal output shows the following commands and results:

```
bash - "p-10-16-0-86 ec2 x" bash - "p-10-16-0-86 ec2 x" Immediate (Javascript (br x)
voclabs:~/environment/1-no-container $ npm start
> start
> node index.js
Leader 4157 is running
Worker 4164 started
Worker 4165 started
GET /api/users - 3
GET /api/users/4 - 4
GET /api/threads - 0
GET /api/posts/in-thread/1 - 0
^C
voclabs:~/environment/1-no-container $
```

Task 3.2: Provisioning the repository

Using Amazon ECR, I'm putting up a location to keep the container image of our message board program. My first step is to locate Elastic Container Registry in the AWS Management Console. Next, I make a brand-new repository called "mb-repo" and set its visibility to private. After that, Amazon ECR verifies that the repository was properly established, giving us a central location to save the container image for our application.



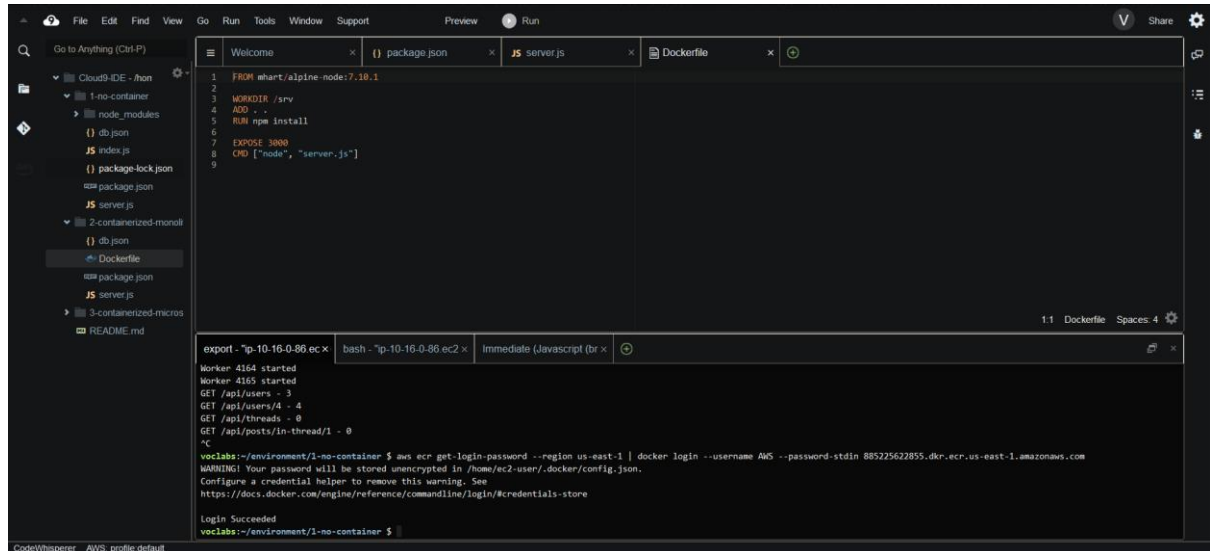
SIT233 – Cloud Computing

Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

Task 3.3: Building and pushing the Docker image

First, I click on "View push commands" in the message window, which opens a pop-up with AWS CLI commands tailored for our repository. I choose the macOS/Linux tab since I'm using AWS Cloud9.

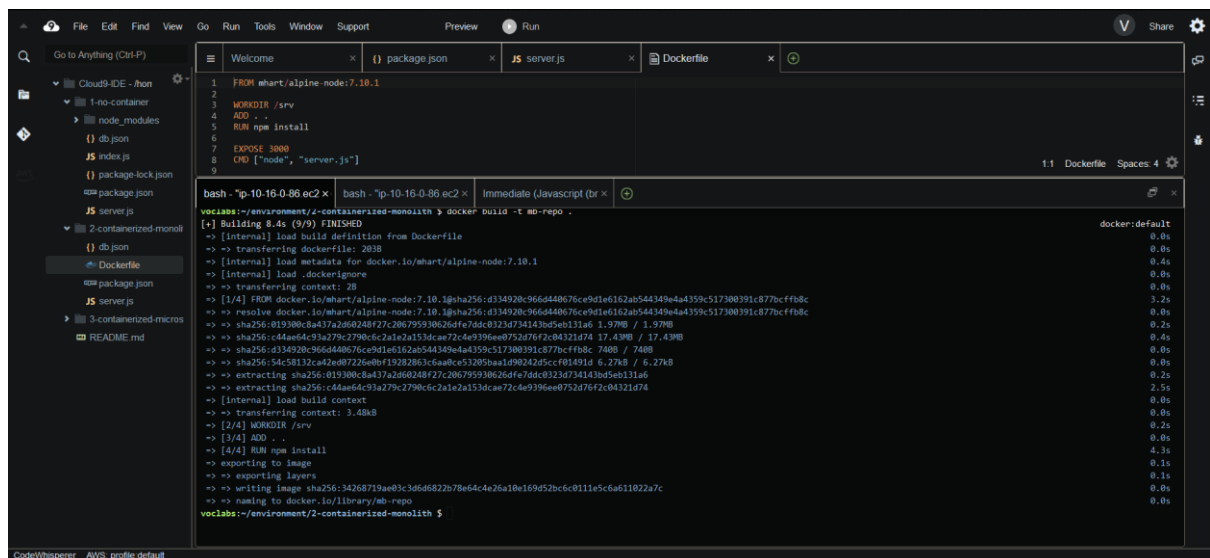


The screenshot shows the AWS Cloud9 IDE interface. The left sidebar displays the file explorer with a tree view of the project structure, including folders like '1-no-container' and '2-containerized-monolith', and files like 'Dockerfile'. The main editor area shows the 'Dockerfile' with the following content:

```
1 FROM whart/alpine-node:7.10.1
2
3 WORKDIR /srv
4 ADD . .
5 RUN npm install
6
7 EXPOSE 3000
8 CMD ["node", "server.js"]
9
```

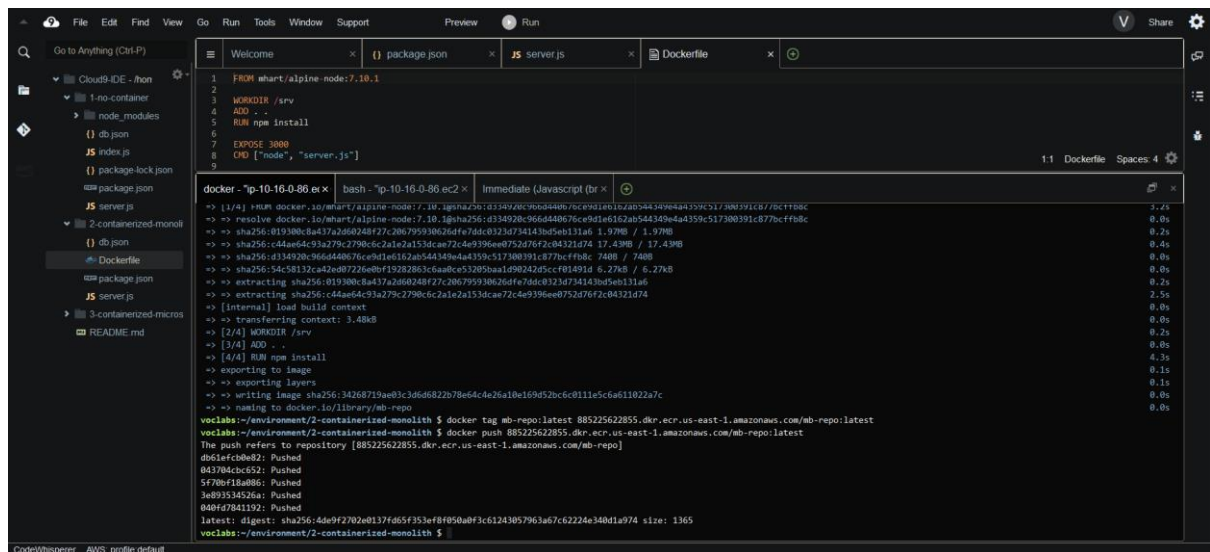
The terminal at the bottom shows the command 'export -p ip-10-16-0-86 ec2' and the output of the 'docker login' command, indicating a successful login to the AWS ECR repository.

Then, I copy and run the command to log in to the Docker client. Once logged in successfully, I move to the AWS Cloud9 tab and navigate to the application folder. Then, I run the command to build the Docker image.



The screenshot shows the AWS Cloud9 IDE interface with the terminal displaying the output of the 'docker build' command. The command executed is 'docker build -t mb-repo .'. The output shows the progress of the build, including the resolution of the Dockerfile, the loading of metadata, and the building of the image. The build is completed successfully, and the image is named 'docker.io/library/mb-repo'.

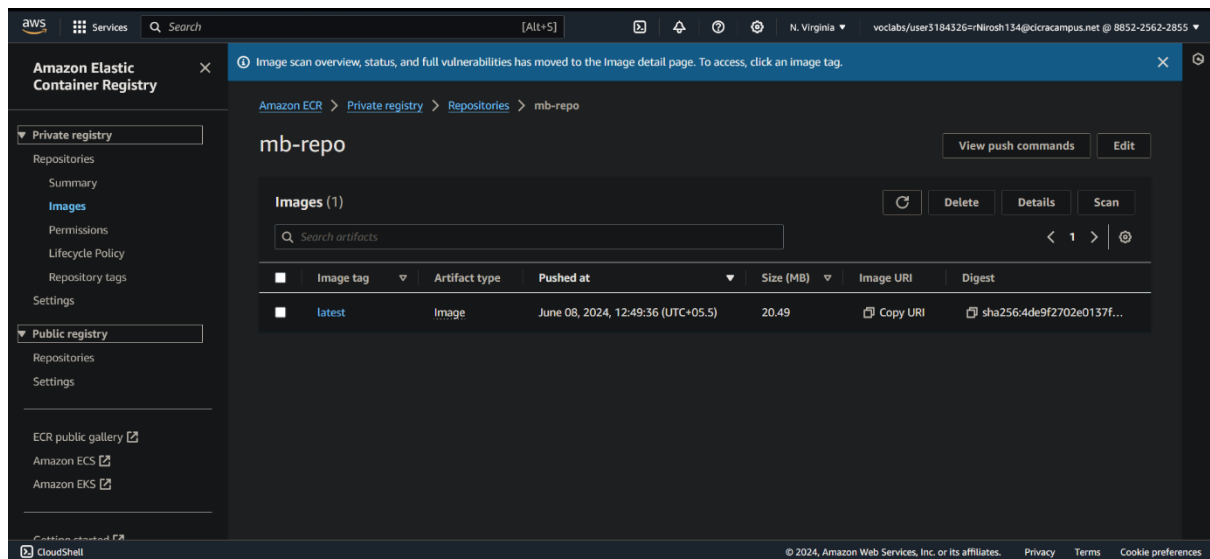
After it completes, I tag the image with the repository URI and push it to the repository by following the commands provided in the pop-up. Each step is executed in the terminal tab.



```
1 FROM mhart/alpine-node:7.10.1
2
3 WORKDIR /srv
4 ADD . .
5 RUN npm install
6
7 EXPOSE 3000
8 CMD ["node", "server.js"]
```

```
docker - "ip-10-16-0-86.ec2" bash - "ip-10-16-0-86.ec2" Immediate (Javascript (br x)
-> [1/4] FROM docker.io/mhart/alpine-node:7.10.1ghsha256:d334926c96d448676ce9d1e1612ad54434964a4359c517300391c877bcffb8c
-> resolve docker.io/mhart/alpine-node:7.10.1ghsha256:d334926c96d448676ce9d1e1612ad54434964a4359c517300391c877bcffb8c
-> sha256:81930b8ba17a2d8d248f27c20679593062dfe7d6c0123d734143b5eb111a6 1.97MB / 1.97MB
-> sha256:c44ae64c93a279c2798c6c2a1e2a153dcae72c4e9396ee0752d76f2c04321d74 17.43MB / 17.43MB
-> sha256:d334926c96d448676ce9d1e1612ad54434964a4359c517300391c877bcffb8c 7408 / 7408
-> sha256:54c58132ca2e07226e0bf19282863c6a0ce53205ba1d90242d5ccf01491d 6.27kB / 6.27kB
-> extracting sha256:01930b8ba17a2d8d248f27c20679593062dfe7d6c0123d734143b5eb111a6
-> extracting sha256:c44ae64c93a279c2798c6c2a1e2a153dcae72c4e9396ee0752d76f2c04321d74
-> [internal] load build context
-> transferring context: 3.48kB
-> [2/4] WORKDIR /srv
-> [3/4] ADD . .
-> [4/4] RUN npm install
-> exporting to image
-> exporting layers
-> writing image sha256:34268719ee83c36e6822b78e64e26a10e169d520c6c011e5c6e611022a7c
-> naming to docker.io/library/mb-repo
voclabs:/environment/2-containerized-monolith $ docker tag mb-repo:latest 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-repo:latest
voclabs:/environment/2-containerized-monolith $ docker push 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-repo:latest
The push refers to repository [885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-repo]
d061efc0be22: Pushed
043704cb652: Pushed
5f70bf18a086: Pushed
3e893534526a: Pushed
0d0bf7041192: Pushed
latest: digest: sha256:4de9f2702e0137f6d5f333ef8f050abf3c61243057963a67c62224e340d1e074 size: 1365
voclabs:/environment/2-containerized-monolith $
```

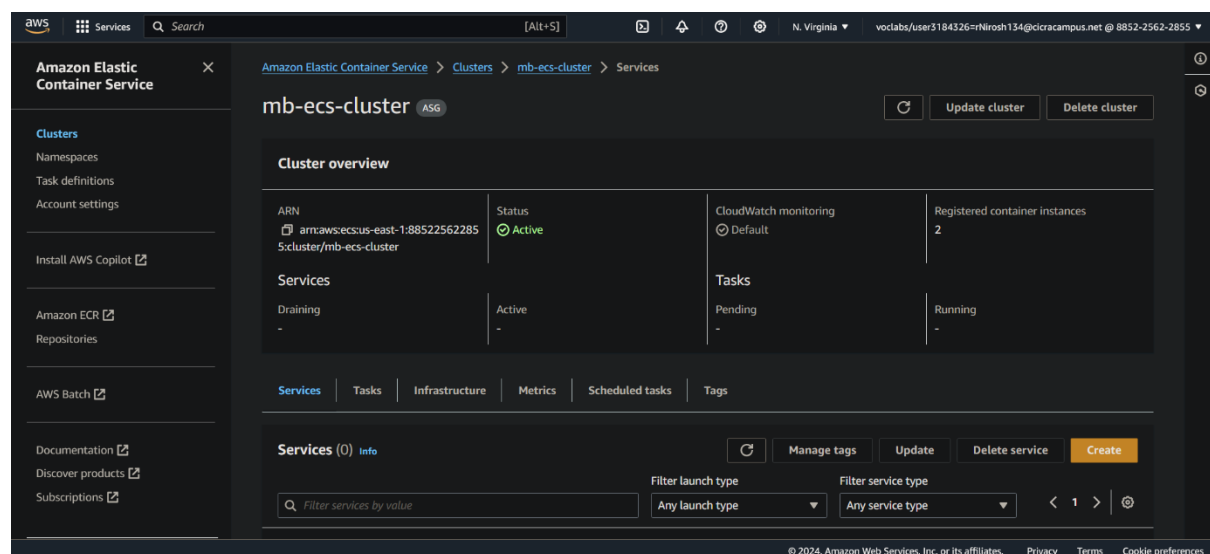
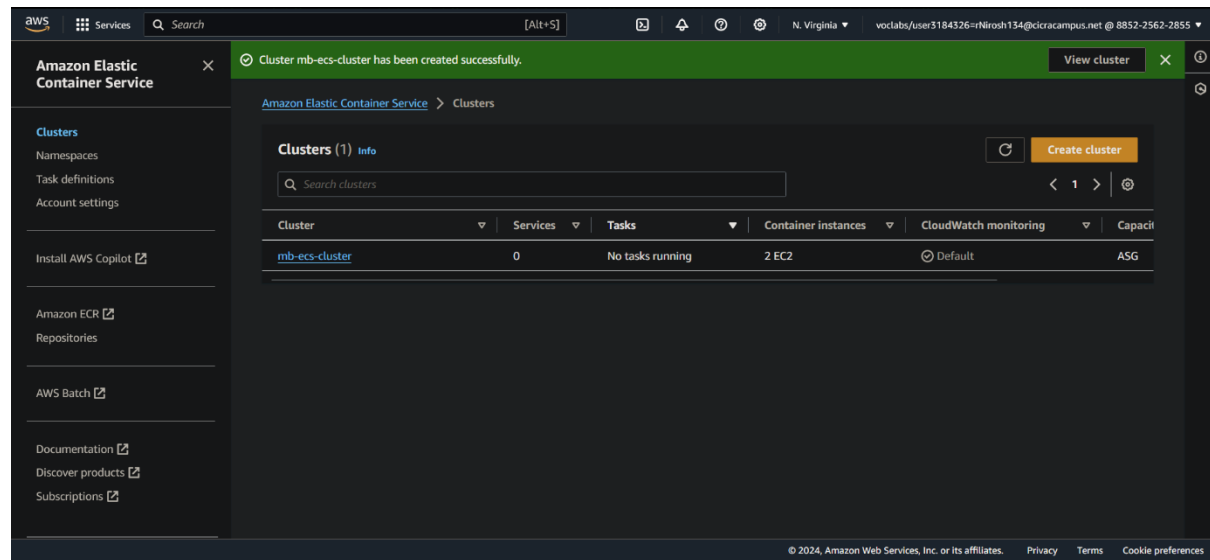
Once pushed, I verify the image in the repository on the Amazon ECR console, ensuring it's successfully uploaded. Finally, I record the image URI for future reference.



Task 4: Deploying the monolith to Amazon ECS

Task 4.1: Creating an Amazon ECS cluster

I start by utilizing the Amazon ECS console to construct an ECS cluster. With particular configurations, like instance type (t2.medium) and desired capacity (2 instances), I set up the cluster to use Amazon EC2 instances.

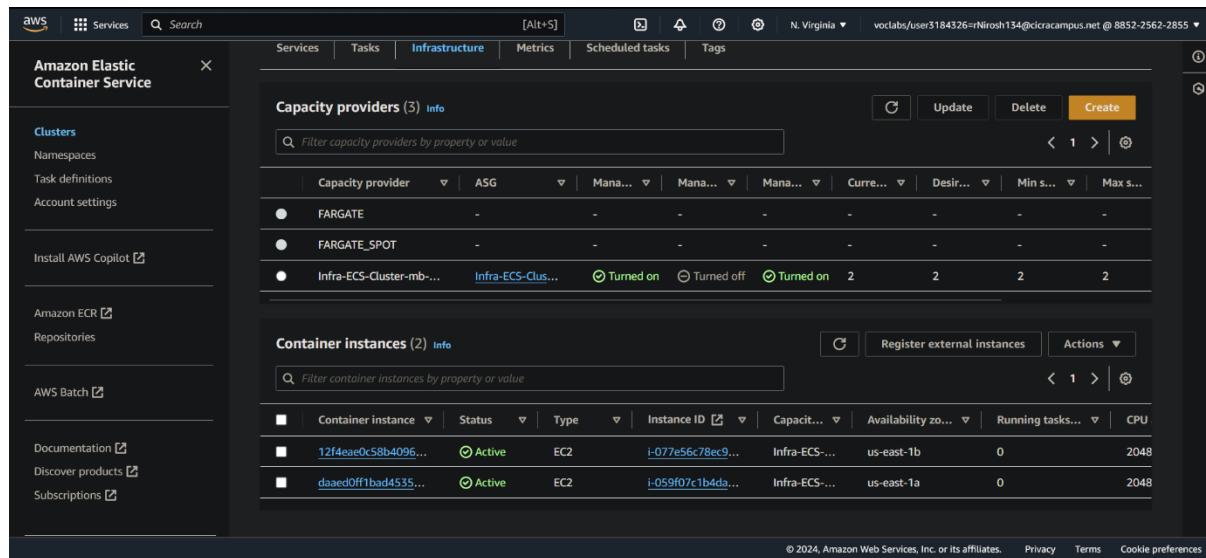


SIT233 – Cloud Computing

Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

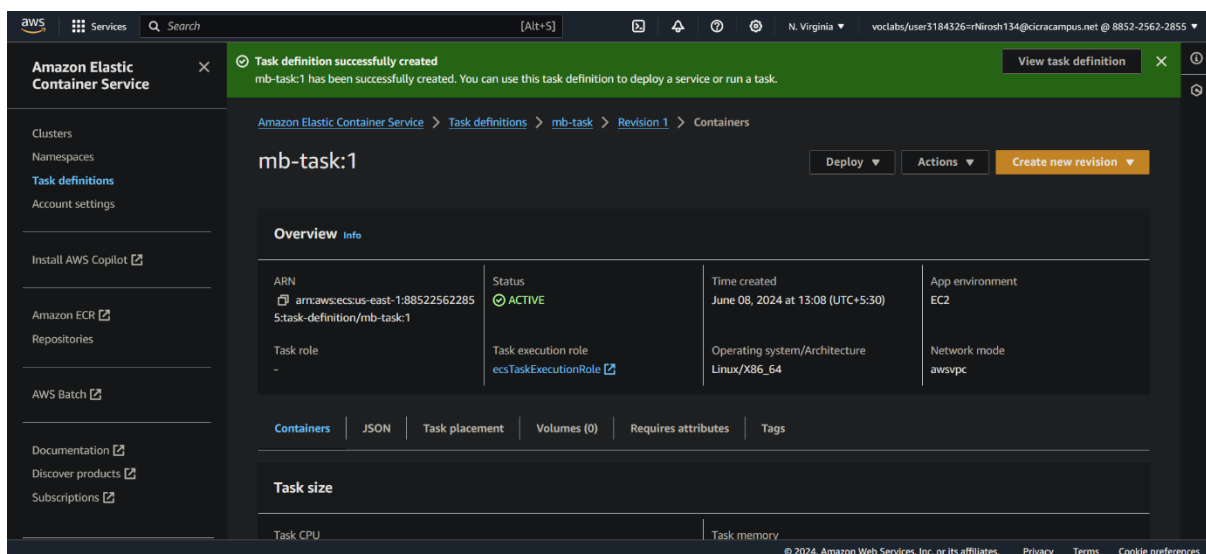
Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

I choose an already-existing security group and make sure that every public subnet is selected. The cluster, known as mb-ecs-cluster, has two EC2 instances and is operational after a short while.



Task 4.2: Creating a task definition for the application container image

I then create a task definition that explains how to use ECS to start a Docker container. I designate a new role for the task's execution, provide the task's CPU and memory requirements, and set up the container's specifics—such as the image URI and port mappings. ECS is instructed on how to deploy my application container by this task definition.



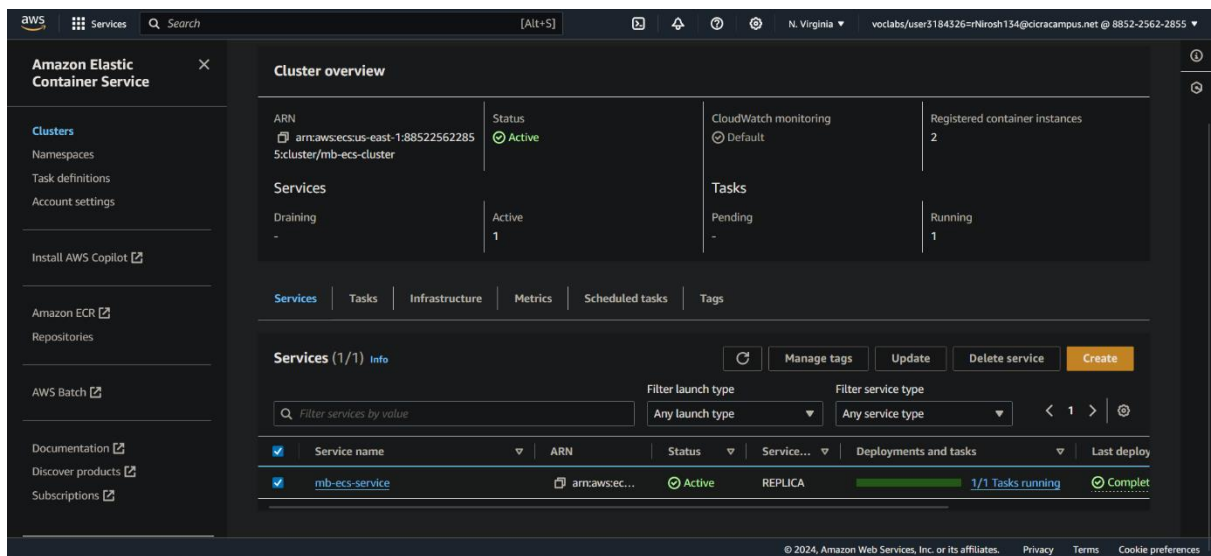
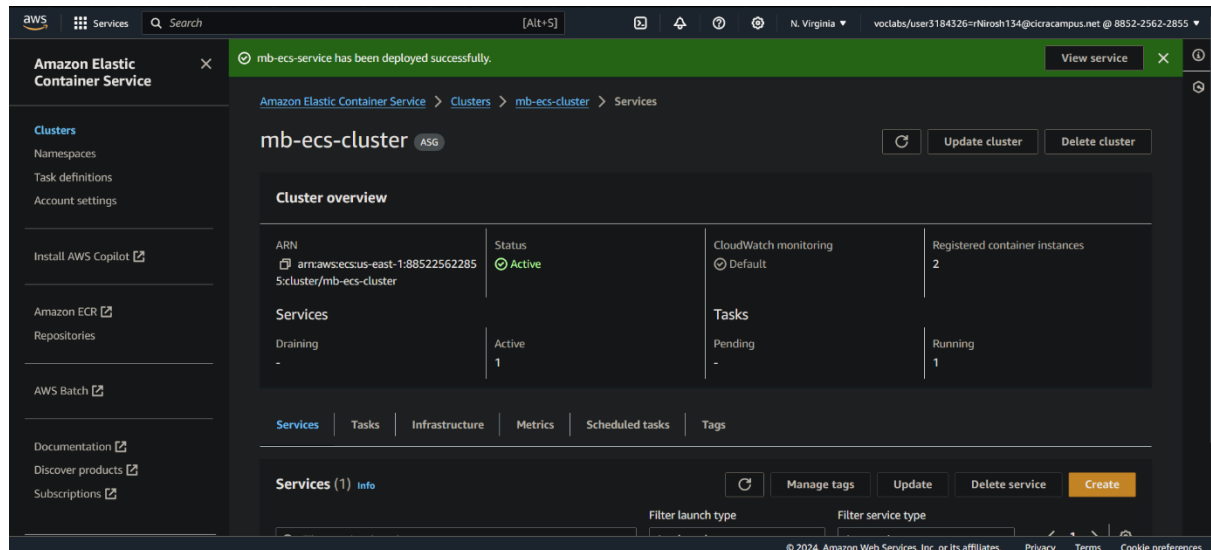
SIT233 – Cloud Computing

Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

Task 4.3: Creating and deploying the service

After setting up the infrastructure, I launch the application by creating an ECS service. I establish networking and security settings, set up an Application Load Balancer, and set the service to use EC2 instances.



I confirm the application is prepared to take requests by checking the service's status and the load balancer's DNS name after it has been created.



Application into Microservices

Task 4.4: Testing the containerized monolith

I test the application's RESTful API methods in order to verify the deployment.

API ready to receive requests

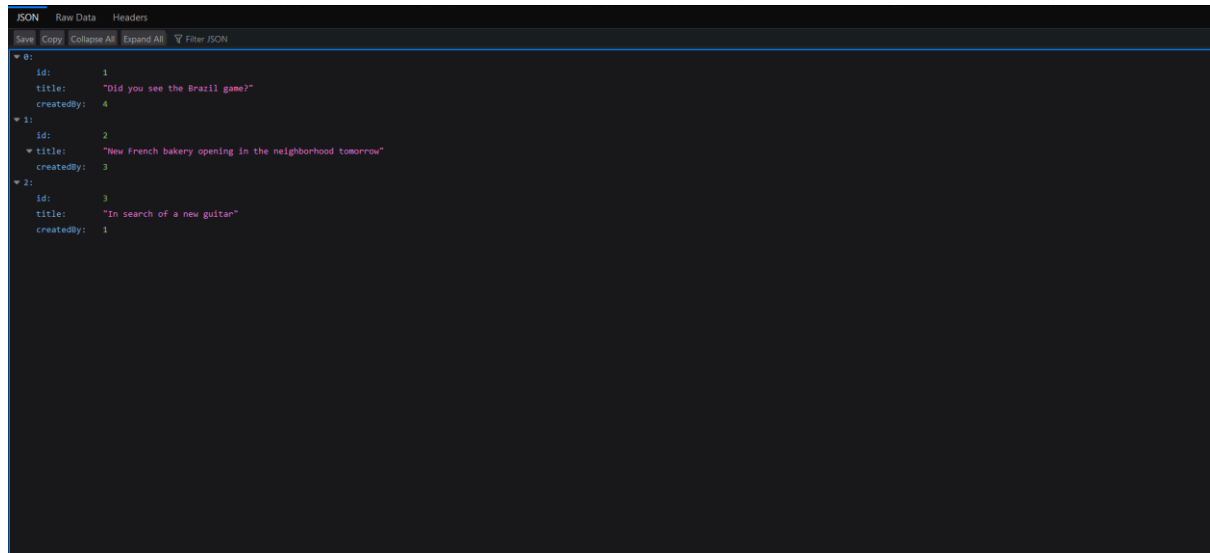
```
JSON  Raw Data  Headers
Save Copy Collapse All Expand All Filter JSON
▼ 0:
  id: 1
  username: "marceline"
  name: "Marceline Singer"
  bio: "Cyclist, musician"
▼ 1:
  id: 2
  username: "finn"
  name: "Finn Alberts"
  bio: "Adventurer and hero, defender of good"
▼ 2:
  id: 3
  username: "pb"
  name: "Paul Barium"
  bio: "Scientist, cake lover"
▼ 3:
  id: 4
  username: "jake"
  name: "Jake Stone"
  bio: "Soccer fan, sky diver"
```

SIT233 – Cloud Computing

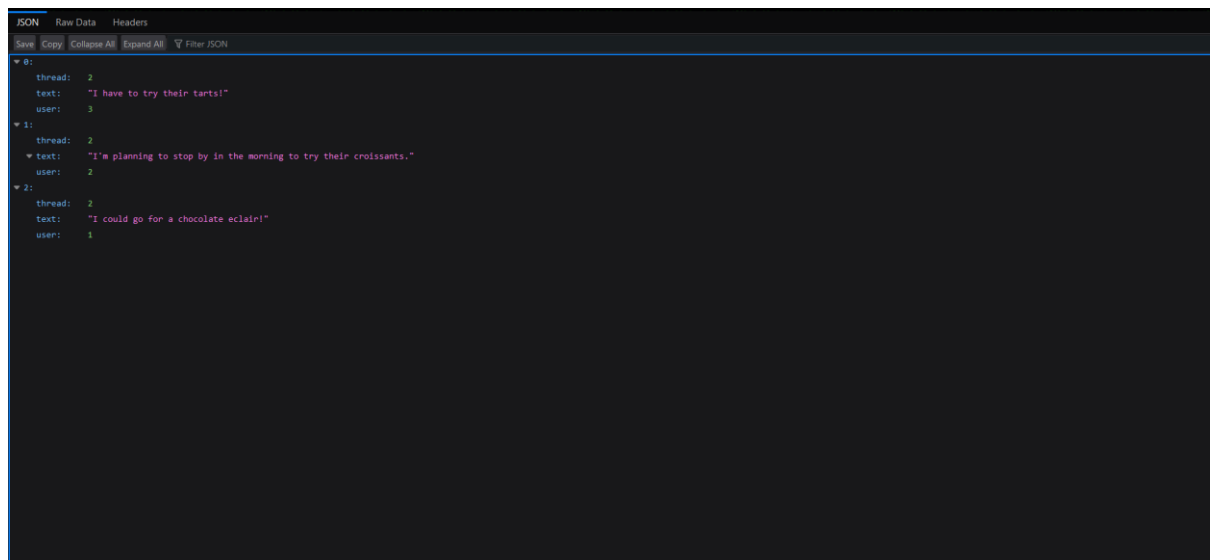
Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

I check that different API endpoints produce the anticipated results, just like when the program ran locally, by using the load balancer's DNS name to access them.



```
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
[
  {
    "id": 1,
    "title": "Did you see the Brazil game?",
    "createdBy": 4
  },
  {
    "id": 2,
    "title": "New French bakery opening in the neighborhood tomorrow",
    "createdBy": 3
  },
  {
    "id": 3,
    "title": "In search of a new guitar",
    "createdBy": 1
  }
]
```



```
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
[
  {
    "thread": 2,
    "text": "I have to try their tarts!",
    "user": 3
  },
  {
    "thread": 2,
    "text": "I'm planning to stop by in the morning to try their croissants.",
    "user": 2
  },
  {
    "thread": 2,
    "text": "I could go for a chocolate eclair!",
    "user": 1
  }
]
```

Task 5: Refactoring the monolith

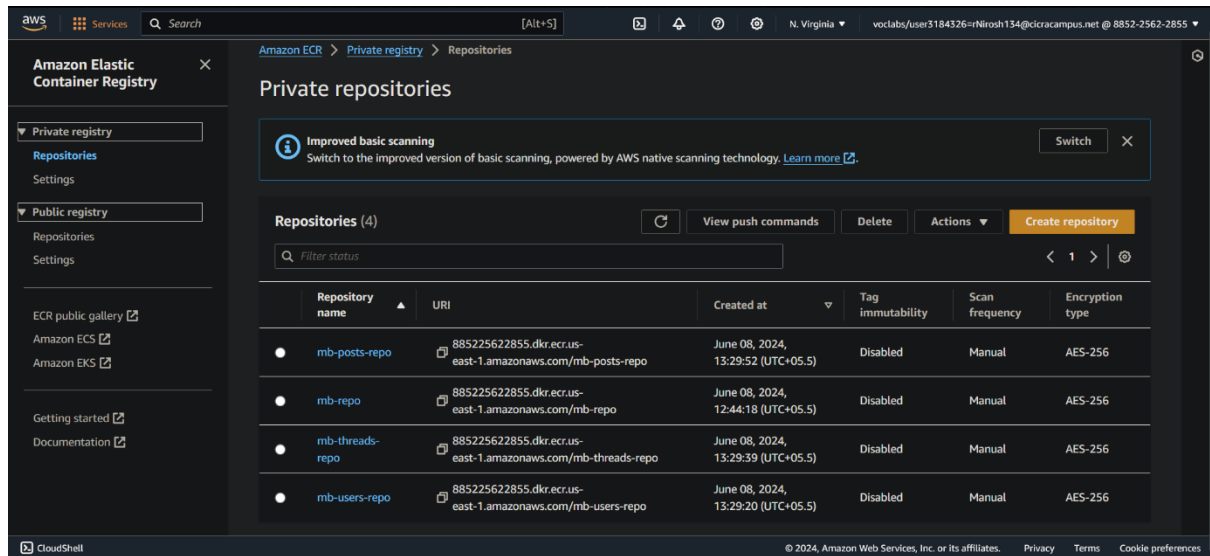
Task 5.1: Reviewing the refactored microservices application

The screenshot shows a Docker terminal window with the following content:

```
FROM mhart/alpine-node:7.10.1
WORKDIR /srv
ADD . .
RUN npm install
EXPOSE 3000
CMD ["node", "server.js"]
```

The terminal output shows the following steps:

```
bash - "ip-10-10-0-86.ec2" bash - "ip-10-10-0-86.ec2" Immediate (Javascript (br x
-> [1/4] RUN docker.io/mhart/alpine-node:7.10.1 sha256:d334920c96d440676ce9d1e61b2ad544349e4a399c517300391c87b0c7f8b0c 3.2s
-> resolve docker.io/mhart/alpine-node:7.10.1 sha256:d334920c96d440676ce9d1e61b2ad544349e4a399c517300391c87b0c7f8b0c 0.0s
-> sha256:819308c8a37a268248f27c206795930626dfe7ddc8323d734143bd5eb11a6 1.97MB / 1.97MB 0.2s
-> sha256:c44ae64c93a279c2790c6c2a1e2a153dca72c4e9396ee0752076f2c04321d74 17.43MB / 17.43MB 0.4s
-> sha256:d334920c96d440676ce9d1e61b2ad544349e4a399c517300391c87b0c7f8b0c 740B / 740B 0.0s
-> sha256:54c58132ca42e07226e0bf19282863c6aafce53205ba1d90242d5ccf01491d 6.27kB / 6.27kB 0.0s
-> extracting sha256:819308c8a37a268248f27c206795930626dfe7ddc8323d734143bd5eb11a6 0.2s
-> extracting sha256:c44ae64c93a279c2790c6c2a1e2a153dca72c4e9396ee0752076f2c04321d74 2.5s
-> [internal] load build context
-> transferring context: 3.48kB 0.0s
-> [2/4] WORKDIR /srv 0.2s
-> [3/4] ADD . . 0.0s
-> [4/4] RUN npm install 4.3s
-> exporting to image 0.1s
-> exporting layers 0.1s
-> writing image sha256:34268719ae83c3d5d6822b78e64c4e26a10e169d52bc6c8111e5c6a611022a7c 0.0s
-> naming to docker.io/library/mb-repo
voclabs:/environment/2-containerized-monolith $ docker tag mb-repo:latest 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-repo:latest
voclabs:/environment/2-containerized-monolith $ docker push 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-repo:latest
The push refers to repository [885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-repo]
db61efcb82: Pushed
043704c0b52: Pushed
5f70bf18a86c: Pushed
3e893934526a: Pushed
040fd7841192: Pushed
latest: digest: sha256:4de9f2702eb177fd5f353ef8f050a0f3c61243057963a67c62224e340d1a974 size: 1365
voclabs:/environment/2-containerized-monolith $
```

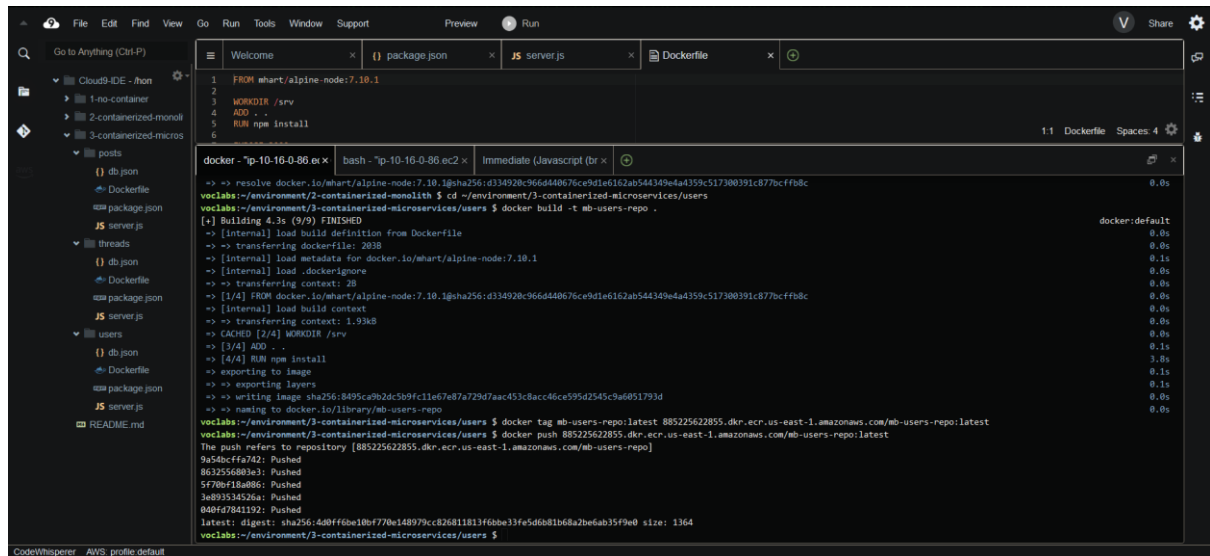


The screenshot displays the Amazon Elastic Container Registry (ECR) Private registries page. The left sidebar shows the navigation menu with options for Private registry, Public registry, ECR public gallery, Amazon ECS, Amazon EKS, Getting started, and Documentation. The main content area shows the Private registries page with a notification for Improved basic scanning. Below the notification, there is a section for Repositories (4) with a search bar and a table listing the repositories.

Repository name	URI	Created at	Tag immutability	Scan frequency	Encryption type
mb-posts-repo	885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-posts-repo	June 08, 2024, 13:29:52 (UTC+05.5)	Disabled	Manual	AES-256
mb-repo	885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-repo	June 08, 2024, 12:44:18 (UTC+05.5)	Disabled	Manual	AES-256
mb-threads-repo	885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-threads-repo	June 08, 2024, 13:29:39 (UTC+05.5)	Disabled	Manual	AES-256
mb-users-repo	885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-users-repo	June 08, 2024, 13:29:20 (UTC+05.5)	Disabled	Manual	AES-256

Application into Microservices

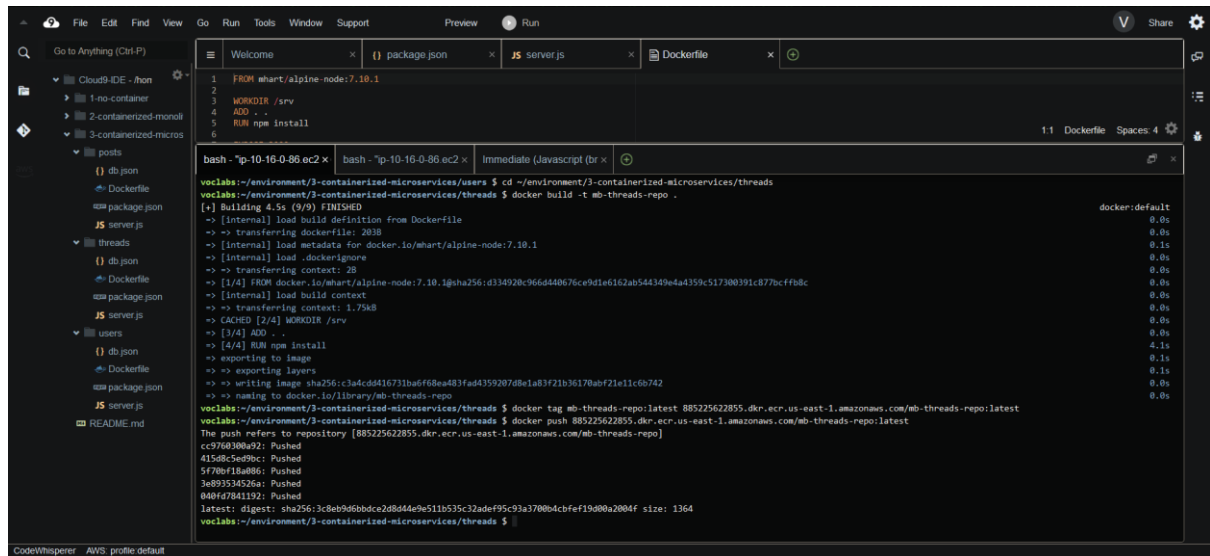
Task 5.3: Building and pushing the images for each microservice

Task 5.3.1: Building and pushing the image for the users microservice

```
1 FROM mhart/alpine-node:7.10.1
2
3 WORKDIR /srv
4 ADD . .
5 RUN npm install
6
```

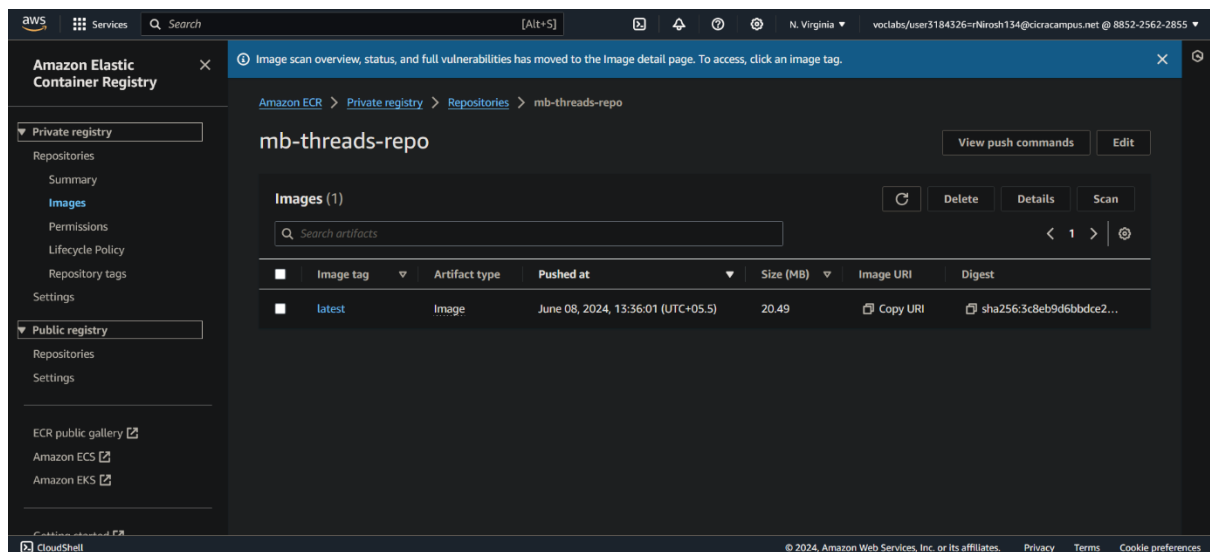
```
docker - "tp-10-16-0-86.ec2" bash - "tp-10-16-0-86.ec2" Immediate (JavaScript (br x)
-> resolve docker.io/mhart/alpine-node:7.10.1 sha256:d34928c96d440676ce9d1e6162ab544349e4a4359c517300391c877bcffb8c 0.0s
voclabs:/environment/2-containerized-monolith $ cd ~/environment/3-containerized-microservices/users
voclabs:/environment/3-containerized-microservices/users $ docker build -t mb-users-repo .
[+] Building 4.3s (9/9) FINISHED
-> [internal] load build definition from Dockerfile 0.0s
-> [internal] transferring Dockerfile: 203B 0.0s
-> [internal] load metadata for docker.io/mhart/alpine-node:7.10.1 0.1s
-> [internal] load .dockerignore 0.0s
-> [internal] transferring context: 2B 0.0s
-> [1/4] FROM docker.io/mhart/alpine-node:7.10.1 sha256:d34928c96d440676ce9d1e6162ab544349e4a4359c517300391c877bcffb8c 0.0s
-> [internal] load build context 0.0s
-> [internal] load metadata for docker.io/mhart/alpine-node:7.10.1 0.0s
-> [internal] transferring context: 1.9kB 0.0s
-> [2/4] WORKDIR /srv 0.0s
-> [3/4] ADD . . 0.1s
-> [4/4] RUN npm install 3.8s
-> exporting to image 0.1s
-> exporting layers 0.1s
-> writing image sha256:8495ca9b2dc5b9fc11e67e87a729d7aac453c8acc46ce595d2545c9a6051793d 0.0s
-> naming to docker.io/library/mb-users-repo 0.0s
voclabs:/environment/3-containerized-microservices/users $ docker tag mb-users-repo:latest 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-users-repo:latest
voclabs:/environment/3-containerized-microservices/users $ docker push 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-users-repo:latest
The push refers to repository [885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-users-repo]
9a54bcffa742: Pushed
863256083e3: Pushed
5f70bf18a086: Pushed
3ab93534526a: Pushed
0a0fd7841192: Pushed
latest: digest: sha256:40ff6be10bf770e148979cc826811813f6bbe33fe5dd881b68a2be6ab35f9e0 size: 1364
voclabs:/environment/3-containerized-microservices/users $
```

Application into Microservices

Task 5.3.2: Building and pushing the image for the threads microservice

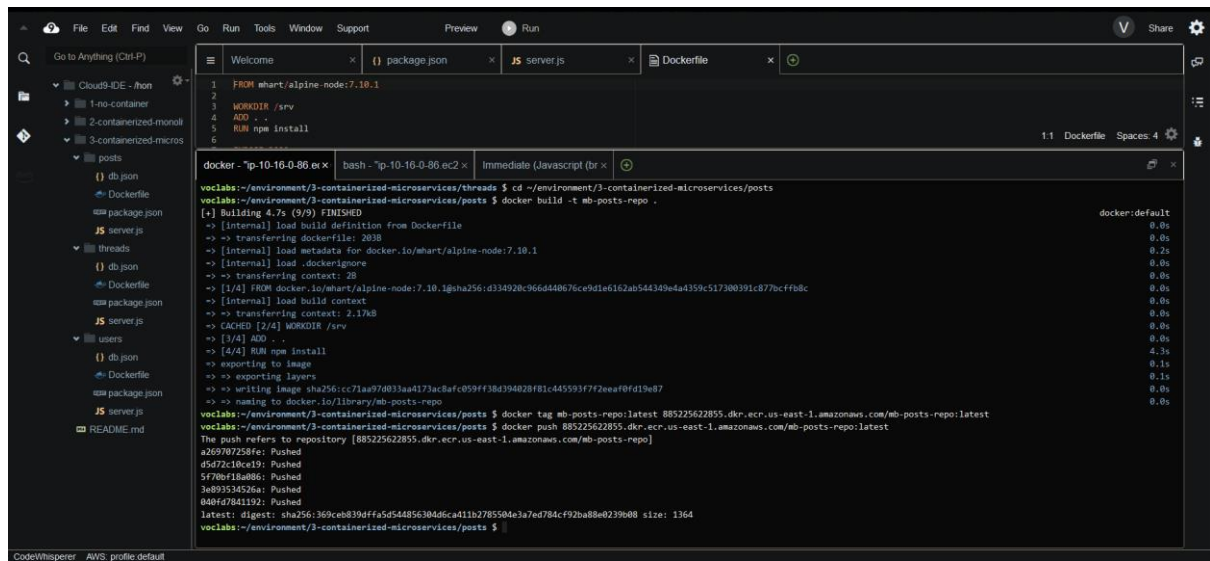
```
1 FROM alpine:node:7.10.1
2
3 WORKDIR /srv
4 ADD . .
5 RUN npm install
6
```

```
bash - "ip-10-16-0-86.ec2" bash - "ip-10-16-0-86.ec2" Immediate (JavaScript) (br x)
voclabs:~/environment/3-containerized-microservices/users $ cd ~/environment/3-containerized-microservices/threads
voclabs:~/environment/3-containerized-microservices/threads $ docker build -t mb-threads-repo .
[+] Building 4.5s (9/9) FINISHED
-> [internal] load build definition from Dockerfile
-> => transferring dockerfile: 283B
-> [internal] load metadata for docker.io/alpine:node:7.10.1
-> [internal] load .dockerignore
-> => transferring context: 2B
-> [1/4] FROM docker.io/alpine:node:7.10.1@sha256:d334920c96d44067fce9d1e612ab344340e4a359c517300391c8770c7fbd6
-> [internal] load build context
-> => transferring context: 1.75kB
-> [2/4] WORKDIR /srv
-> [3/4] ADD . .
-> [4/4] RUN npm install
-> exporting to image
-> => exporting layers
-> => writing image sha256:3a4cd416731ba6f6bea483fd4359207d8e1a83f21b36170abf21e11c6b742
-> naming to docker.io/library/mb-threads-repo
voclabs:~/environment/3-containerized-microservices/threads $ docker tag mb-threads-repo:latest 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-threads-repo:latest
voclabs:~/environment/3-containerized-microservices/threads $ docker push 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-threads-repo:latest
The push refers to repository [885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-threads-repo]
cc976030ba92: Pushed
4156b55e99cc: Pushed
5f70b71ba086: Pushed
3e893334526a: Pushed
040fd7841192: Pushed
latest: digest: sha256:3a4cd416731ba6f6bea483fd4359207d8e1a83f21b36170abf21e11c6b742 size: 1364
voclabs:~/environment/3-containerized-microservices/threads $
```



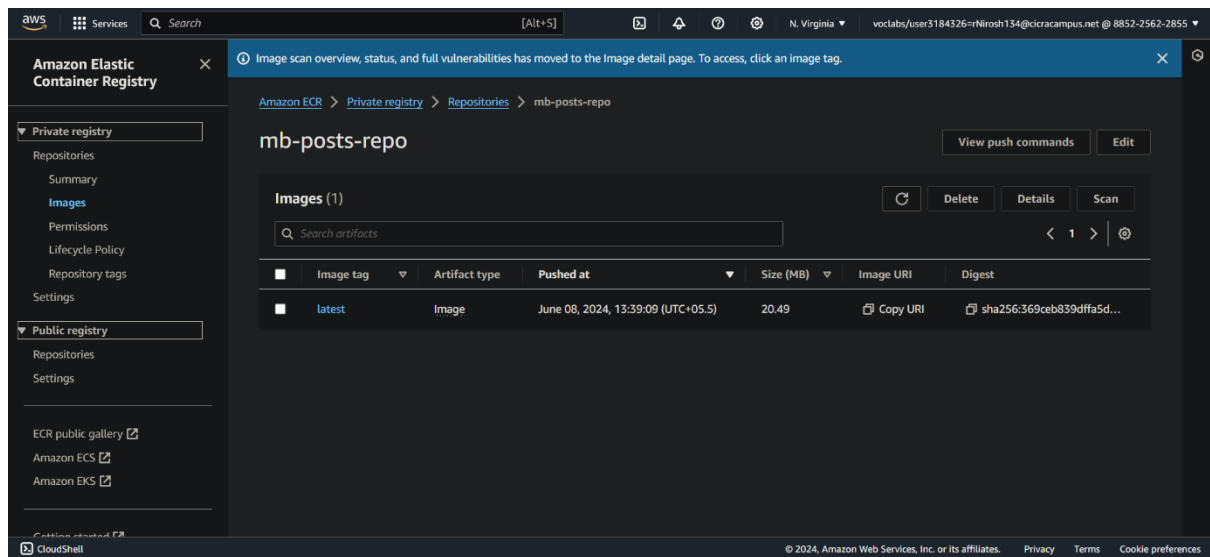
Application into Microservices

Task 5.3.3: Building and pushing the image for the posts microservice



```
1 FROM mhart/alpine-node:7.10.1
2
3 WORKDIR /srv
4 ADD . .
5 RUN npm install
6
```

```
docker - "p-10-16-0-86 ex x" bash - "p-10-16-0-86 ec2 x" Immediate (javascript (br x)
voclabs:~/environment/3-containerized-microservices/threads $ cd ~/environment/3-containerized-microservices/posts
voclabs:~/environment/3-containerized-microservices/posts $ docker build -t mb-posts-repo .
[+] Building 4.7s (9/9) FINISHED
-> [internal] load build definition from Dockerfile
-> [internal] load metadata for docker.io/mhart/alpine-node:7.10.1
-> [internal] load .dockerignore
-> [internal] load build context
-> [1/4] FROM docker.io/mhart/alpine-node:7.10.1@sha256:d334020c966d440676ce9d1e6162ab544349e4a4359c517300391c877bcff8dc
-> [internal] load build context
-> [internal] load metadata for docker.io/mhart/alpine-node:7.10.1
-> [2/4] WORKDIR /srv
-> [3/4] ADD . .
-> [4/4] RUN npm install
-> exporting image
-> writing image sha256:cc71aa97d03aa4173ac8f8c859f38d394028f81c445593f722eaf0fd19e87
-> naming to docker.io/library/mb-posts-repo
voclabs:~/environment/3-containerized-microservices/posts $ docker tag mb-posts-repo:latest 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-posts-repo:latest
voclabs:~/environment/3-containerized-microservices/posts $ docker push 885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-posts-repo:latest
The push refers to repository [885225622855.dkr.ecr.us-east-1.amazonaws.com/mb-posts-repo]
a269707258fe: Pushed
d5d72c10ce19: Pushed
5f70bf18a086: Pushed
3a89334526a: Pushed
040fd7841192: Pushed
latest: digest: sha256:369ceb839dffa5d5448563b04dca411b2785504e3a7ed784cf92ba89e0239e08 size: 1364
voclabs:~/environment/3-containerized-microservices/posts $
```



Task 6: Deploying the containerized microservices

Task 6.1: Creating a task definition for each microservice

Task 6.1.1: Creating a task definition for the users microservice

Since every microservice operates on its own, I must write unique task definitions for each one.

I start by going to the Task definitions section of the AWS Management Console, searching for Elastic Container Service. I make a new task definition called mb-users-task, with 0.5 vCPU and 1GB of memory as the job size, and I set EC2 as the infrastructure need. I give the users container image's URI and configure the container's details, designating it mb-users-container. The container port is now set to 3000. I construct the task definition and verify its successful creation after setting up every parameter.

Task definition successfully created
mb-users-task:1 has been successfully created. You can use this task definition to deploy a service or run a task.

Amazon Elastic Container Service > Task definitions > mb-users-task > Revision 1 > Containers

mb-users-task:1

Deploy Actions Create new revision

Overview

ARN arn:aws:ecs:us-east-1:885225622855:task-definition/mb-users-task:1	Status ACTIVE	Time created June 08, 2024 at 13:45 (UTC+5:30)	App environment EC2
Task role -	Task execution role ecsTaskExecutionRole	Operating system/Architecture Linux/X86_64	Network mode awsvpc

Containers JSON Task placement Volumes (0) Requires attributes Tags

Task size

Task CPU	Task memory
----------	-------------

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Task 6.1.2: Creating a task definition for the posts microservice

I then carry out the same steps for the posts microservice. I make a new task definition called mb-posts-task in the Task definitions section with the following configurations: 1GB of RAM, 0.5 vCPU, and container information, such as the URI for the posts container image, are all included. Moreover, 3000 is the container port setting. I create the task definition and see the success message after everything is configured.

The screenshot displays the AWS Management Console for the Amazon Elastic Container Service. A green notification banner at the top states: "Task definition successfully created. mb-posts-task:1 has been successfully created. You can use this task definition to deploy a service or run a task." The breadcrumb navigation shows the path: Amazon Elastic Container Service > Task definitions > mb-posts-task > Revision 1 > Containers. The main heading is "mb-posts-task:1" with buttons for "Deploy", "Actions", and "Create new revision".

The "Overview" tab is selected, showing a table with the following details:

ARN	Status	Time created	App environment
arn:aws:ecs:us-east-1:885225622855:task-definition/mb-posts-task:1	ACTIVE	June 08, 2024 at 13:47 (UTC+5:30)	FARGATE

Below the table, the "Task role" is listed as "-", and the "Task execution role" is "ecsTaskExecutionRole". The "Operating system/Architecture" is "Linux/X86_64", and the "Network mode" is "awsvpc".

At the bottom, the "Task size" section is partially visible, showing "Task CPU" and "Task memory".

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Task 6.1.3: Creating a task definition for the threads microservice

Lastly, I create the threads microservice task specification. I put it up with the identical CPU, RAM, and container settings, giving it the name mb-threads-task. I build the task definition and verify its creation after inputting the URI for the threads container image and configuring the port to 3000.

The screenshot displays the AWS Management Console for the Amazon Elastic Container Service (ECS). A green notification banner at the top states: "Task definition successfully created. mb-threads-task:1 has been successfully created. You can use this task definition to deploy a service or run a task." The breadcrumb navigation shows the path: Amazon Elastic Container Service > Task definitions > mb-threads-task > Revision 1 > Containers. The main heading is "mb-threads-task:1". Below this, there are buttons for "Deploy", "Actions", and "Create new revision". The "Overview" tab is selected, showing a table of metadata:

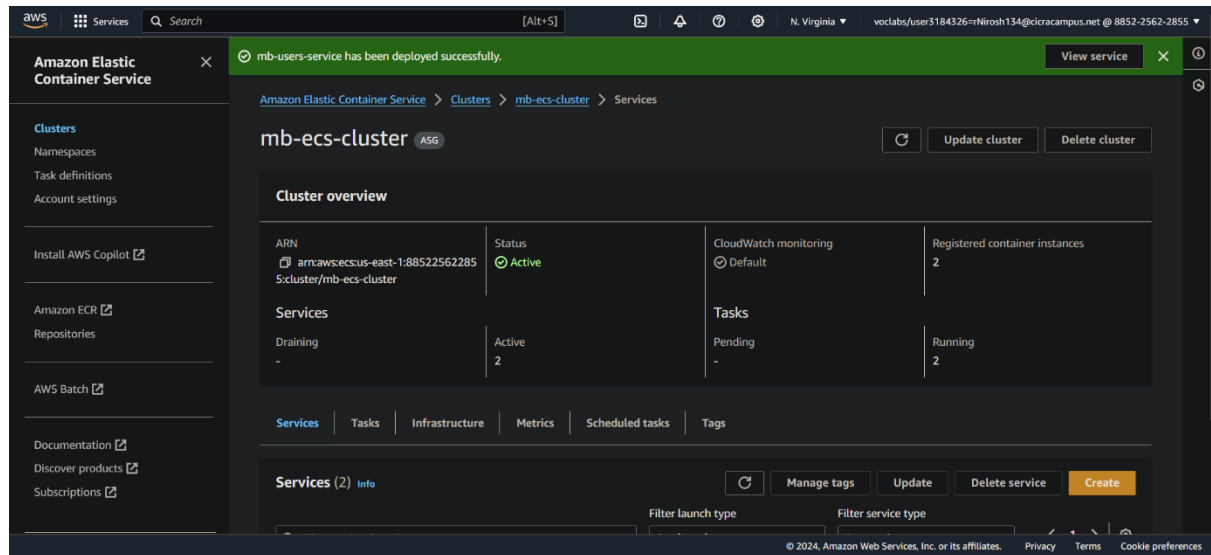
Overview	
ARN arn:aws:ecs:us-east-1:885225622855:task-definition/mb-threads-task:1	Status ACTIVE
Task role -	Task execution role ecsTaskExecutionRole
Time created June 08, 2024 at 13:49 (UTC+5:30)	App environment FARGATE
Operating system/Architecture Linux/X86_64	Network mode awsvpc

Below the metadata table, there are tabs for "Containers", "JSON", "Task placement", "Volumes (0)", "Requires attributes", and "Tags". The "Task size" section is partially visible at the bottom, showing "Task CPU" and "Task memory". The footer of the console indicates "© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences".

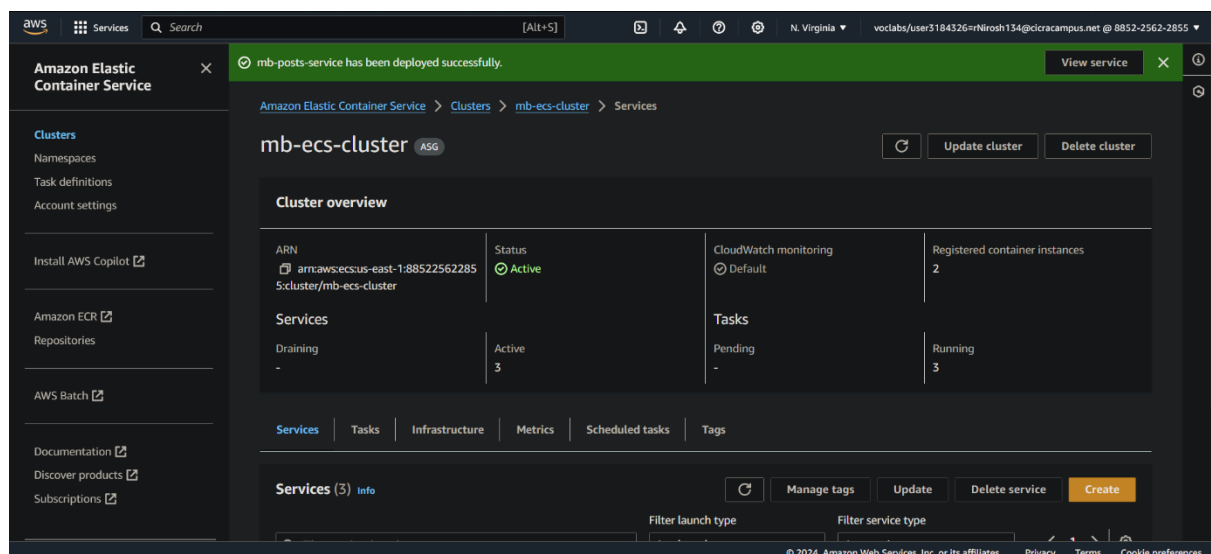
Task 6.2: Creating and deploying the services

After completing the job descriptions, I proceed to the microservices' deployment as ECS services.

I start by going to the ECS console's Clusters section and choosing my mb-ecs-cluster. I make a new service called mb-users-service and set the launch type to EC2 for the mb-users-task.



In the networking section, I set up a new target group called mb-users-target with the path pattern /api/users* and evaluation order 1, using an existing security group and load balancer. I wait for the creation of the service while I save.

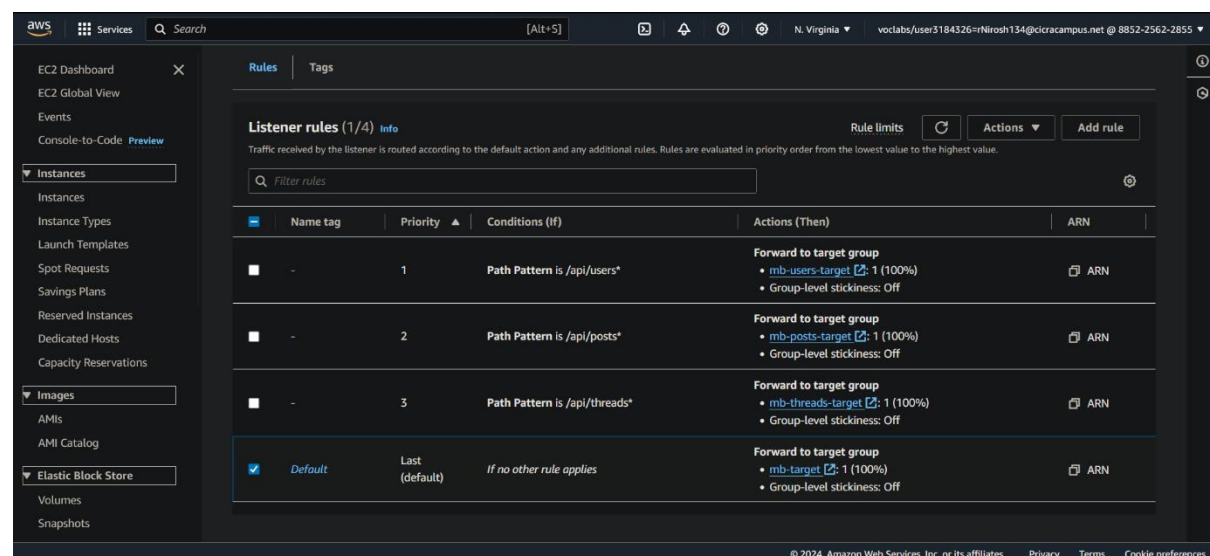
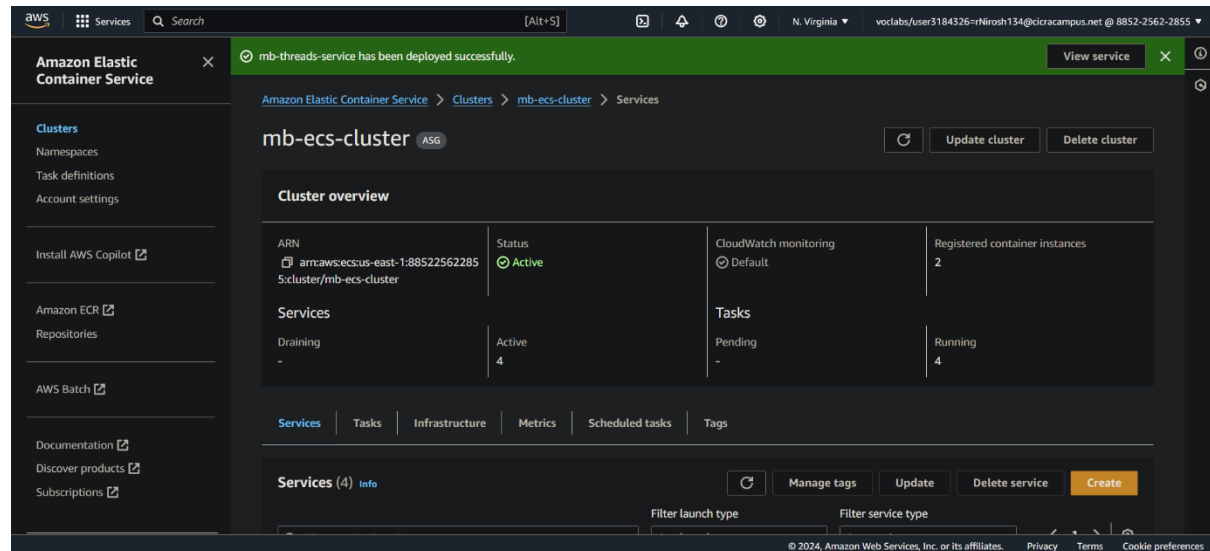


SIT233 – Cloud Computing

Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

I then carry out the same procedure for the mb-posts-task, renaming the service mb-posts-service and generating a new target group called mb-posts-target with evaluation order 2 and the path pattern /api/posts*. I wait for the development of this service.

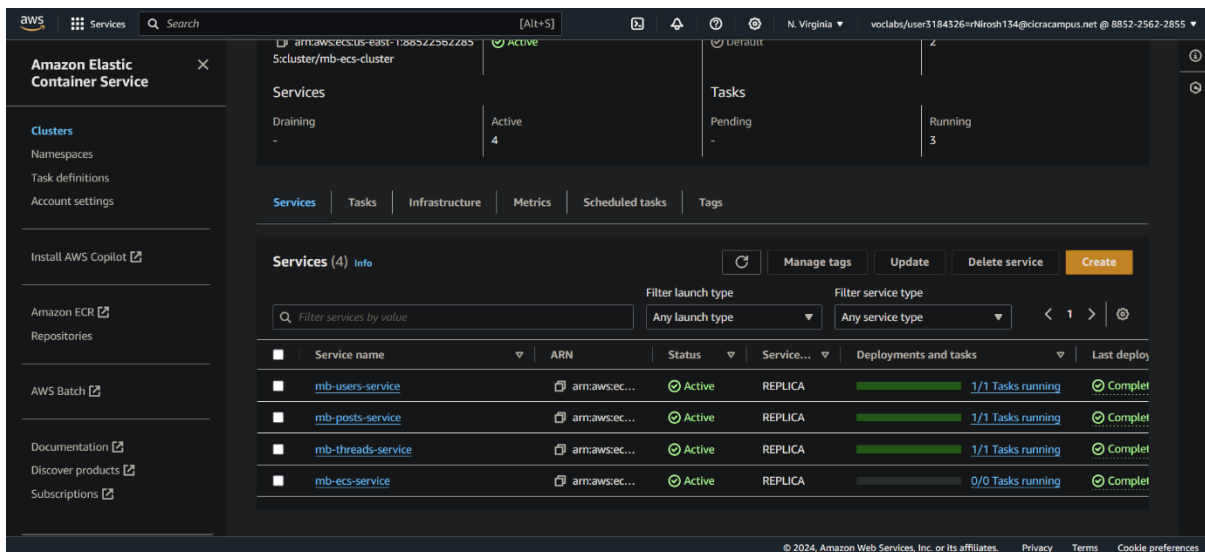
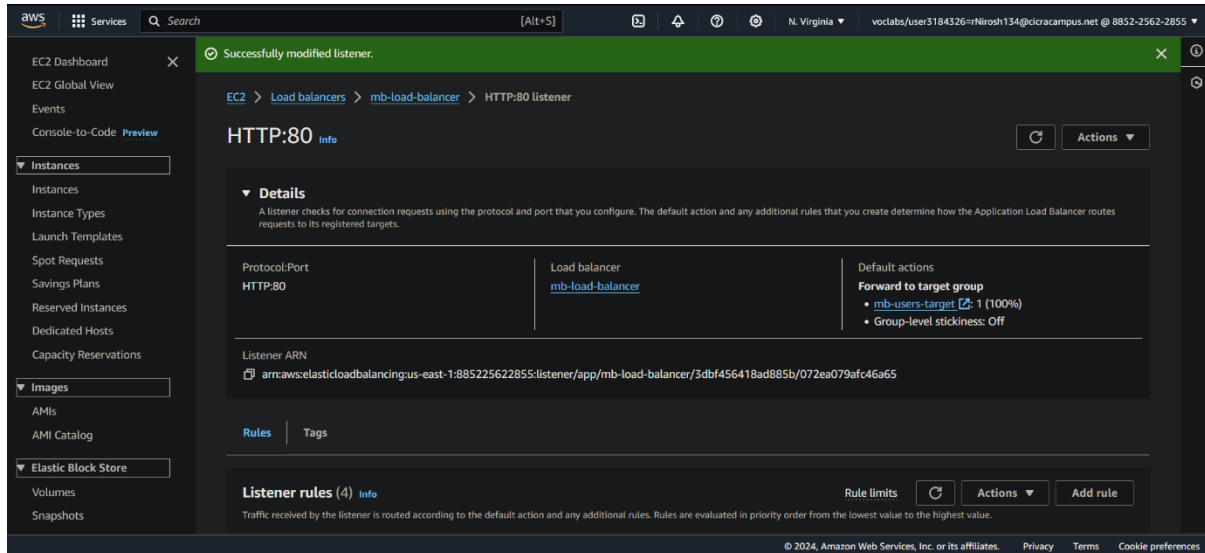


SIT233 – Cloud Computing

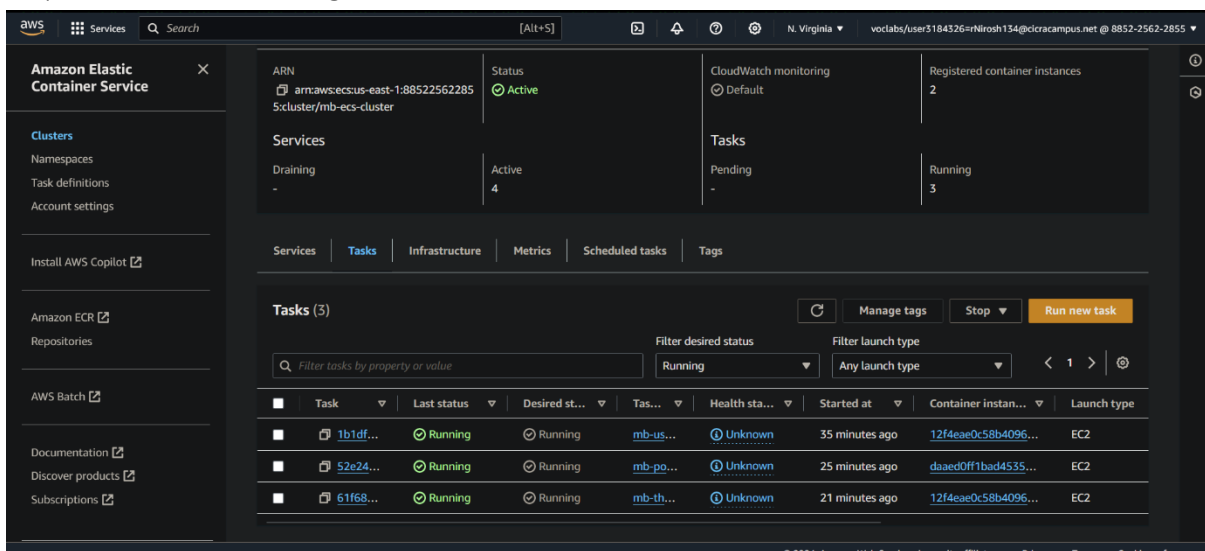
Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Name: Nirosh Ravindran
CICRA Student ID: BSCP|CS|63|134

Lastly, I make a target group called mb-threads-target with the path pattern /api/threads* and evaluation order 3, as well as a service for the mb-threads-task that I call mb-threads-service. I wait for the development of the service to finish once more.



I go back to the ECS panel, pick any service, and check the load balancer rules after establishing every service. I make the necessary adjustments to the default rule, forwarding requests to mb-users-target, and save it.



Task 6.3: Validating the deployment

I initially stopped the monolithic container's ECS service to make sure it was no longer processing requests in order to verify the deployment. I then use a web browser and type the DNS name of the load balancer.

I confirm the user's microservice is managing the default requests when the notification "Ready to receive requests" shows.

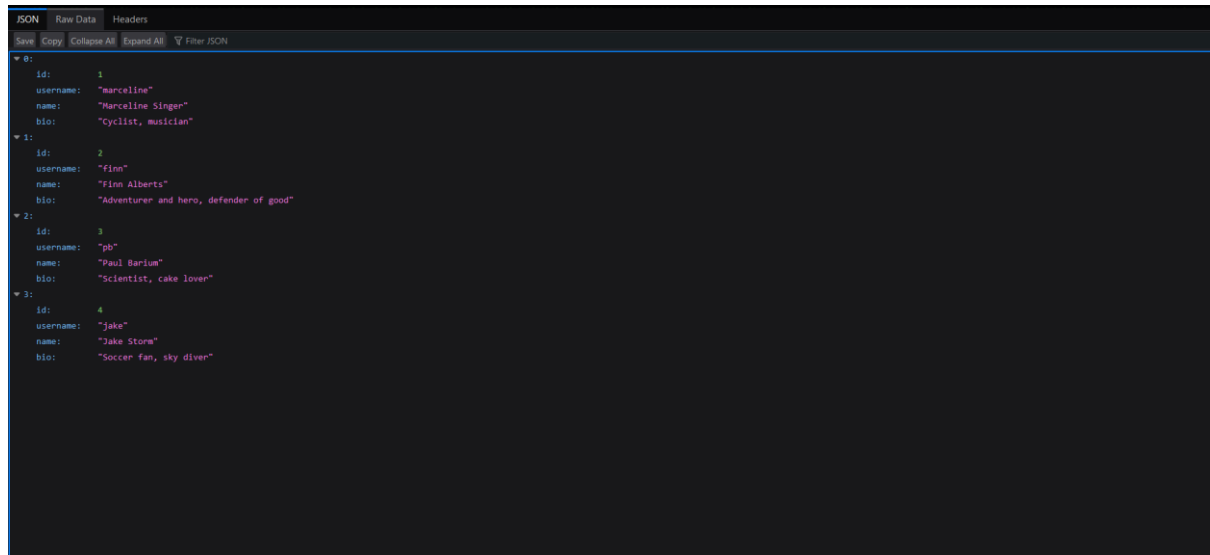
A screenshot of a terminal window with a black background. The text "Ready to receive requests" is displayed in a small, light-colored font at the top left of the terminal area.

Ready to receive requests

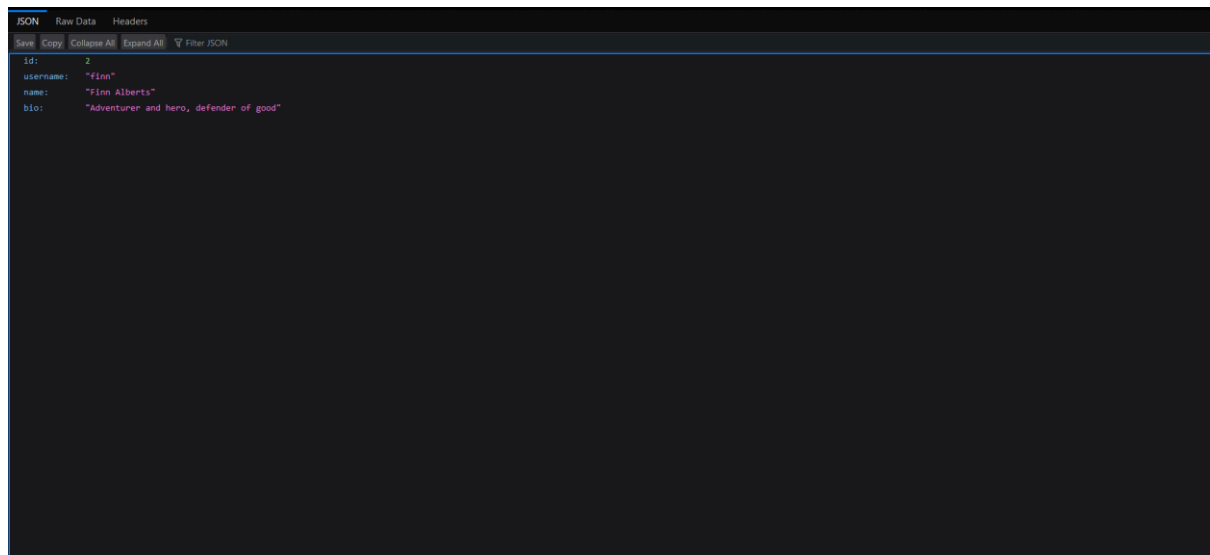
A screenshot of a terminal window with a black background. The text "API ready to receive requests" is displayed in a small, light-colored font at the top left of the terminal area.

API ready to receive requests

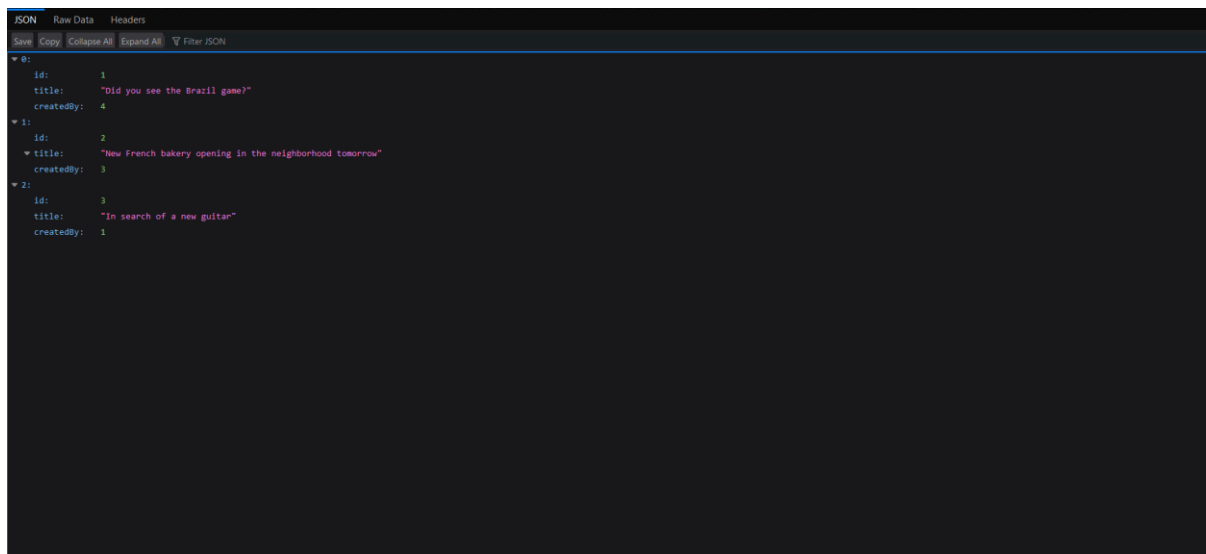
I then use a variety of URLs to test the particular API pathways in the browser. I verify that the expected output is delivered for every path (/api/users, /api/users/2, /api/threads, /api/posts/in-thread/2) to ensure that the microservices are processing requests appropriately.



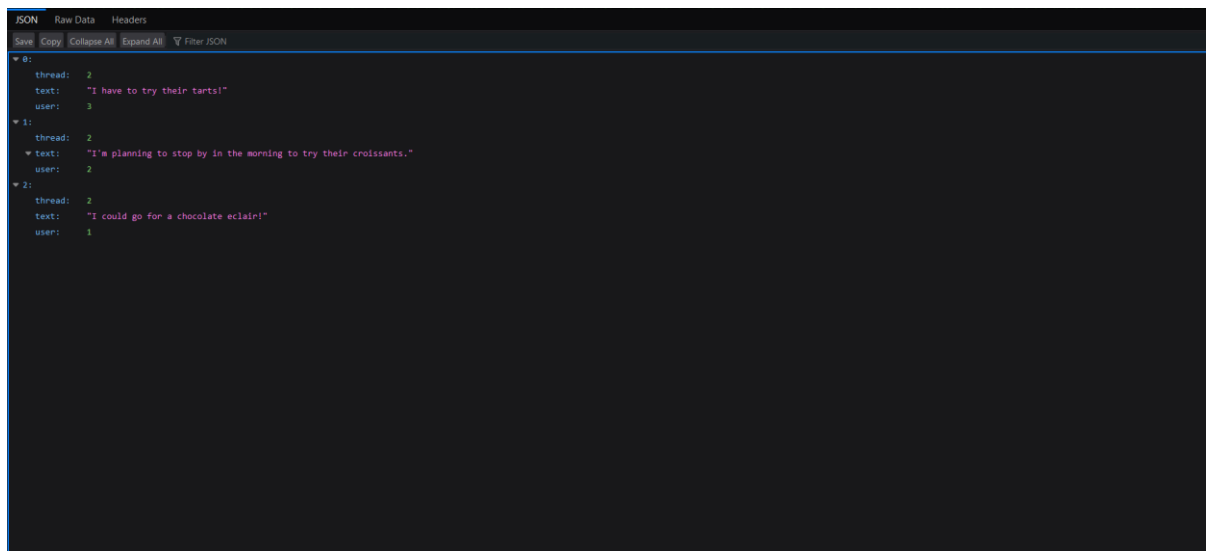
```
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
▼ 0:
  id: 1
  username: "marceline"
  name: "Marceline Singer"
  bio: "Cyclist, musician"
▼ 1:
  id: 2
  username: "finn"
  name: "Finn Alberts"
  bio: "Adventurer and hero, defender of good"
▼ 2:
  id: 3
  username: "pb"
  name: "Paul Barium"
  bio: "Scientist, cake lover"
▼ 3:
  id: 4
  username: "jake"
  name: "Jake Storm"
  bio: "Soccer fan, sky diver"
```



```
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
id: 2
username: "finn"
name: "Finn Alberts"
bio: "Adventurer and hero, defender of good"
```



```
{
  "0": {
    "id": 1,
    "title": "Did you see the Brazil game?",
    "createdBy": 4
  },
  "1": {
    "id": 2,
    "title": "New French bakery opening in the neighborhood tomorrow",
    "createdBy": 3
  },
  "2": {
    "id": 3,
    "title": "In search of a new guitar",
    "createdBy": 1
  }
}
```



```
{
  "0": {
    "thread": 2,
    "text": "I have to try their tarts!",
    "user": 3
  },
  "1": {
    "thread": 2,
    "text": "I'm planning to stop by in the morning to try their croissants.",
    "user": 2
  },
  "2": {
    "thread": 2,
    "text": "I could go for a chocolate eclaire!",
    "user": 1
  }
}
```

In order to confirm that it produces the "Invalid request" message, I also test an invalid path (/xxx).

I successfully deploy and validate the containerized microservices application on AWS ECS by following these procedures.

Not Found

Grade

My grade is 70/75

(Optional) Module 13 - Guided Lab 1: Breaking a Monolithic Node.js Application into Microservices

Due No Due Date Points 100 Submitting an external tool

AWS 02:16 ▶ Start Lab ■ End Lab ⓘ AWS Details ⓘ Details ↺ Reset ✕

Submit Submission Report Grades

EN_US application's task definition.

58. On the same screen where you created *mb-task*, choose **Deploy**, then choose **Create service**

59. In the **Environment** section, configure the following options:

- For **Compute options**, choose **Launch type**.
- For **Launch type**, choose **EC2**.
- For **Service name** choose *mb-ecs-service*
- Expand the **Networking** section, and configure the following options:
 - For **Security group**, choose **Use an existing security group**.
 - From the **Security group name** dropdown list, select the security group that has **ECSSG** in the name.

Total score 70/75

[Task 3A] ECR Repo created	5/5
[Task 3B] Image pushed to repo	5/5
[Task 4A] mb-ecs-cluster created	5/5
[Task 4B] Application Load Balancer created	5/5
[Task 4D] ECS Service configured	0/5

◀ Previous Next ▶